



INSTITUTO POLITÉCNICO NACIONAL

**ESCUELA SUPERIOR DE INGENIERÍA MECÁNICA Y
ELÉCTRICA
UNIDAD CULHUACÁN**

Reporte de proyecto

*Ecosistema multiagente para la generación y validación de
circuitos analógicos CA/CD a partir de lenguaje natural*

PRESENTAN:

ANTONIO DE JESUS TORRES MARTÍNEZ

DIEGO JESUS GUTIERREZ DELGADO

ASESOR:

M. en C. RICARDO ISRAEL CALZADA SALAS



Ciudad de México 27 de septiembre de 2026

Índice

Introducción	1
Planteamiento del Problema	1
Objetivo General	2
Objetivos Específicos	2
Estado del Arte	2
AnalogCoder	2
AnalogAgent	3
CircuitLM	3
Justificación	4
Límites y Alcances	5
1. Marco Teórico	6
1.0.1. Qué es un agente	6
1.0.2. Que es un prompt	6
1.0.3. Modelos de lenguaje grande	7
1.0.4. LangGraph	8
1.0.5. LangFuse	8
1.0.6. Spice (Simulation Program with Integrated Circuit Emphasis)	8
1.0.7. PySpice	9
1.0.8. Generacion Aumentada por Recuperación (RAG)	9
1.0.9. Sistemas multiagente y orquestación	10
1.0.10. Arquitectura de software: principios y patrones de coordinación	11
1.0.11. Separacion de intereses	11
1.0.12. Ausencia de estado en el componente a escalar	11
1.0.13. La ausencia de estado entre solicitudes convive con una exigencia opuesta dentro de cada solicitud	12
1.1. El diseño de circuitos analógicos como proceso iterativo	12
1.2. Síntesis del marco teórico	13
2. Diseño y desarrollo del proyecto	14
2.1. Propuesta de Diseño	14
2.2. Premisas y fundamento de diseño	15

2.2.1.	El problema de diseño y el papel del SRS	15
2.2.2.	Premisas de diseño	15
2.3.	Descomposición funcional y casos de uso	17
2.3.1.	Vista funcional	17
2.3.2.	Modelo de casos de uso	18
2.4.	Arquitectura del sistema (modelo C4)	19
2.4.1.	Nivel 1 — Contexto	19
2.4.2.	Nivel 2 — Contenedores	20
2.4.3.	Nivel 3 — Componentes	21
2.5.	Comportamiento dinámico: flujo de generación	22
2.6.	Calidad de la solución	25
2.7.	Seguridad de la solución	27
2.8.	Criterios de aceptación de la propuesta	29
2.9.	Síntesis del diseño	30
2.10.	Introducción	30
2.11.	Código fuente del proyecto	30
2.12.	Pantallas del prototipo funcional	31
2.13.	Análisis de seguridad de la implementación	33
2.13.1.	Controles implementados	33
2.13.2.	Correspondencia con las categorías de riesgo OWASP	35
2.13.3.	Riesgos residuales	35
3.	Pruebas y Análisis de Resultados	37
4.	Conclusiones y Trabajo Futuro	38
	Referencias	39
A.	Diagramas de casos de uso	41

Índice de figuras

2.1. Diagrama funcional del sistema.	18
2.2. Diagrama de casos de uso: flujo de simulación y validación.	19
2.3. C4 Nivel 1: Diagrama de contexto.	20
2.4. C4 Nivel 2: Diagrama de contenedores.	21
2.5. C4 Nivel 3: Diagrama de componentes.	22
2.6. Diagrama de secuencia del flujo de generación.	24
2.7. Pantalla de inicio de sesión del prototipo, con la ruta de solución del pipeline de generación.	32
2.8. Espacio de trabajo autenticado: solicitud en lenguaje natural, resumen operativo, actividad reciente y archivos generados.	33
A.1. Diagrama de casos de uso: acceso y gestión de sesiones.	41
A.2. Diagrama de casos de uso: generación de circuito.	42
A.3. Diagrama de casos de uso: decisión del agente curador.	42

Introducción

Planteamiento del Problema

A diferencia del diseño digital, que se apoya en abstracciones estandarizadas y flujos de automatización maduros, el diseño de circuitos analógicos descansa en el conocimiento experto, el razonamiento heurístico y un ajuste iterativo guiado por simulación [1]. Ese ciclo —proponer un circuito, simularlo, revisar los resultados y volver a ajustar— se repite numerosas veces antes de obtener un diseño que cumpla las metas, y validar un diseño completo puede exigir largos periodos de iteración [2]. Buena parte de ese conocimiento reside en la experiencia de quien diseña y no en un procedimiento que pueda seguirse paso a paso, lo que vuelve el diseño analógico difícil tanto de automatizar como de aprender.

Los Modelos de Lenguaje Grande (LLM) abrieron la posibilidad de describir un circuito en lenguaje natural y recibir una propuesta de diseño de forma automática. Al llevarla a la práctica, sin embargo, aparecen limitaciones documentadas. Estos modelos alucinan componentes, violan restricciones físicas del circuito y producen salidas eléctricamente inconsistentes [3], por lo que sus diseños rara vez funcionan en el primer intento, ya que el modelo no comprueba por sí mismo si lo que generó cumple.

Los trabajos publicados se reparten entre dos líneas. La primera diseña a nivel de transistor, atado a un proceso de fabricación específico descrito por su *Process Design Kit* (PDK), y emplea la simulación SPICE como realimentación durante el diseño; es la línea más desarrollada [1], [4], [5]. La segunda trabaja a nivel de placa, con componentes comerciales, y verifica el resultado mediante reglas eléctricas [2], [3], en parte porque no todo componente comercial cuenta con un modelo SPICE utilizable, lo que impide simular cualquier diseño de forma uniforme. La simulación SPICE a nivel de placa ha comenzado a aparecer, pero en forma de banco de prueba que compara el diseño generado contra un circuito de referencia construido por expertos [6], y no como una señal que el propio sistema utilice para ajustar el diseño durante su ejecución. Entre los trabajos revisados no se encontró un sistema generativo de varios agentes que use la simulación SPICE como realimentación dentro del lazo de diseño, con un curador que decida el ajuste, orientado a circuitos analógicos CA/CD con componentes comerciales. Ese es el punto del espacio de diseño que este proyecto se propone ocupar.

Objetivo General

Desarrollar un ecosistema multiagente basado en LLM y aprendizaje por refuerzo para la generación y validación de circuitos analógicos CA/CD con componentes comerciales a partir de descripciones en lenguaje natural, empleando la simulación en ngspice como mecanismo de validación.

Objetivos Específicos

1. **Desarrollar** el ecosistema multiagente (orquestador, master-workers, curador, escritura y shell) para la correcta comunicación de los módulos de cada agente mediante LangGraph y conexiones mediante API's y sistemas externos.
2. **Entrenar e implementar** el agente curador para: indexar, recuperar topologías y reglas de diseño validadas y evitar que los demás agentes alucinen, esto mediante un aprendizaje RAG.
3. **Integrar** ngspice mediante PySpice, para cerrar el lazo de realimentación que permite ajustar los valores de los componentes de forma iterativa.
4. **Diseñar** la arquitectura para la gestión y alojamiento de los módulos (cliente web, ecosistema multiagente, conexión a LLM's), utilizando React, TypeScript y Docker.

Estado del Arte

La automatización y el diseño generativo de circuitos analógicos mediante inteligencia artificial han cobrado especial relevancia en años recientes. Para situar el aporte de este trabajo, resulta indispensable analizar las metodologías propuestas en la literatura técnica reciente.

AnalogCoder

En primer lugar, **AnalogCoder** [4] introdujo la propuesta de diseñar circuitos tratándolos como código ejecutable; en lugar de pedirle al modelo de lenguaje que describa un circuito en lenguaje natural, se le instruye para que escriba un script en Python capaz de generarlo. Al convertir el diseño en un programa ejecutable se reduce significativamente la ambigüedad de la síntesis y el netlist resultante puede simularse de inmediato. El sistema funciona sin necesidad de reentrenar el modelo base y comprueba recursivamente cada propuesta con una secuencia de verificaciones por simulación SPICE, validando que el circuito simule sin nodos flotantes, que los puntos de operación en corriente continua sean correctos y que la respuesta

dinámica o de frecuencia cumpla con las especificaciones. Cuando una verificación falla, la traza de error regresa al modelo para orientar el siguiente intento de corrección. No obstante, esta aproximación coordina todo el flujo con un solo agente y se limita a nivel de transistor sobre un proceso de fabricación específico.

AnalogAgent

Por su parte, **AnalogAgent** [1] reparte esta carga de trabajo mediante un esquema de colaboración multiagente. Su contribución central consiste en la gestión y destilación del conocimiento acumulado durante los fallos: cuando un intento de diseño resulta fallido, el sistema procesa el error para traducirlo en una regla heurística explícita que se incorpora a una memoria auto-evolutiva y se recupera en iteraciones futuras, reduciendo el número total de llamadas al modelo. Adicionalmente, el estudio documenta una limitación típica de los agentes monolíticos basados en modelos de lenguaje: a medida que el historial de chat crece con cada intento de corrección, el resumen de contexto tiende a diluir u omitir detalles e instrucciones técnicas críticas. Esta estrategia de acumular conocimiento para guiar los pasos subsiguientes influye directamente en la concepción del agente curador de este trabajo, aunque mantiene la limitación de operar a nivel de transistor.

CircuitLM

Finalmente, **CircuitLM** [3] comparte con este proyecto el nivel de abstracción a nivel de componentes y módulos comerciales a partir de especificaciones de lenguaje natural. Este sistema emplea una tubería multiagente dedicada a la identificación de componentes, la consulta de hojas de especificaciones y la estructuración del esquema. No obstante, se orienta a electrónica digital y embebida —microcontroladores como Arduino o ESP32— y difiere del presente trabajo en su método de validación: en lugar de integrar simulación SPICE en el lazo, verifica la topología con un motor determinista de reglas eléctricas y un modelo de lenguaje supervisor. Sus propios autores señalan la verificación con simulador en el lazo como una limitación pendiente, justamente la señal de realimentación que este proyecto incorpora.

Ubicación de este Proyecto

Junto a los desarrollos mencionados, otras propuestas complementan el panorama del área. **AnalogSAGE** [5] introduce memoria estratificada y mecanismos de autoevolución a la síntesis a nivel de transistor; **PCBSchemaGen** [2] extiende la verificación estática de reglas al ámbito de tarjetas de circuito impreso reales; y **HWE-Bench** [6] provee un entorno

de evaluación comparativa que mide el rendimiento de netlists generados contrastándolos con diseños de referencia de expertos a través de simulaciones SPICE. Adicionalmente, se registran contribuciones como **ADO-LLM** [7] para la optimización bayesiana en el diseño físico y **SPICEPilot** [8] para la generación asistida de scripts de control.

Al contrastar las soluciones se observa que ninguno de los antecedentes recopilados integra simultáneamente los cuatro rasgos propuestos en este ecosistema. Mientras que los sistemas orientados a transistores explotan la simulación en lazo cerrado y el ajuste paramétrico iterativo, no contemplan el uso de componentes comerciales. Por el contrario, los enfoques comerciales evitan la simulación dinámica en el lazo de optimización o la reducen a una métrica de prueba externa. Este proyecto une la retroalimentación física por simulación de los flujos a nivel de transistor con el diseño a nivel de placa mediante componentes comerciales, implementando una arquitectura multiagente coordinada por un curador entrenado por aprendizaje por refuerzo.

Justificación

Automatizar el diseño de circuitos analógicos tiene valor práctico porque su ciclo de propuesta, simulación y ajuste descansa en el conocimiento experto y no puede seguirse como un procedimiento mecánico. Los modelos de lenguaje que generan diseños a partir de lenguaje natural simplifican la entrada al proceso, pero producen diseños eléctricamente inconsistentes que requieren corrección manual o, cuando los sistemas verifican internamente, lo hacen a nivel de transistor o mediante reglas eléctricas, no mediante simulación SPICE dentro del lazo de diseño.

En la práctica, a partir de una descripción en lenguaje natural el sistema genera un netlist con componentes comerciales, lo simula en ngspice [9], extrae sus métricas eléctricas y ajusta los valores de los componentes de forma iterativa hasta que el diseño cumpla las metas especificadas. El ciclo de ajuste lo ejecuta el propio ecosistema; el usuario recibe el netlist simulado junto con las métricas y el historial de iteraciones.

Los destinatarios directos son ingenieros y estudiantes que trabajen con circuitos analógicos CA/CD contruidos a partir de componentes comerciales con modelo SPICE disponible —resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales de catálogo— y que requieran una herramienta que no solo genere sino que también valide mediante simulación, sin depender de un nodo tecnológico propietario ni de herramientas de licencia costosa.

Lo que se propone es trabajar con componentes comerciales y, a la vez, usar la simulación SPICE como señal de ajuste dentro del lazo de diseño, coordinado por varios

agentes con un curador. Se apoya en herramientas de uso libre —LangGraph [10], PySpice [11] y ngspice [9]—, desplegadas en la nube y sin dependencia de licencias. Desde la ingeniería en computación.

Este proyecto implementa un ecosistema multiagente, que integra tareas mediante procesamiento paralelo y obedece el comportamiento eléctrico, siendo una herramienta mas acertada en las generación y validación de circuitos mediante el uso de LLM's.

Límites y Alcances

El sistema abarca el recorrido completo desde una descripción en lenguaje natural hasta un netlist simulado en ngspice. Del lado del cliente, gestiona el registro y el inicio de sesión, los espacios de trabajo y las claves con las que el usuario envía sus solicitudes. Del lado del servidor, lee la descripción y extrae de ella los parámetros eléctricos, divide los circuitos de varias etapas en sub-bloques, calcula los valores de los componentes, genera el código PySpice y el netlist, simula en ngspice, lee las métricas, ajusta los valores de forma iterativa a través del curador y entrega al usuario el netlist final junto con sus métricas y el historial de iteraciones.

El sistema trabaja únicamente con circuitos analógicos CA/CD formados por componentes comerciales que admiten un modelo SPICE, como resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por su macromodelo. Un componente sin un modelo utilizable queda fuera de su alcance, ya que sin modelo no hay simulación y, por tanto, no hay señal de realimentación. El sistema no diseña la disposición física de la placa (PCB), no realiza síntesis a nivel de transistor para un proceso de fabricación específico (PDK) ni genera esquemáticos gráficos. Por último, los valores numéricos que aparecen en los requisitos son metas de la primera versión y pueden ajustarse durante la evaluación experimental.

Capítulo 1

Marco Teórico

El sistema propuesto se sitúa en el cruce de cuatro dominios técnicos: el diseño de circuitos analógicos, los modelos de lenguaje, la Generación Aumentada por Recuperación y la simulación física de circuitos, articulados sobre una arquitectura de software distribuida. Este capítulo expone los conceptos que sostienen cada uno de esos dominios y, sobre todo, las relaciones entre ellos, porque la aportación del proyecto no reside en ninguno por separado, sino en su integración dentro de un lazo de generación y validación. La exposición sigue el recorrido de los datos en el sistema: desde la descripción en lenguaje natural hasta el netlist validado.

1.0.1. Qué es un agente

En ciencias de la computación, un agente es una entidad computacional autónoma provista de mecanismos de percepción sobre un entorno, capacidad de razonamiento/decisión para alcanzar objetivos y una interfaz de acción (ejecución de herramientas o código). Un sistema multiagente (MAS) distribuye una meta compleja en agentes especializados que colaboran e intercambian mensajes estructurados.

El ecosistema implementa agentes de rol acotado (tales como el Agente Orquestador, Agente de Cálculo/Workers, Agente Curador, Agente de Escritura y Agente Shell). Esto permite aplicar el principio de descomposición ("divide y vencerás"), delimitando la carga de razonamiento para cada etapa física del circuito.

1.0.2. Que es un prompt

Un prompt es la secuencia inicial de texto, directivas, restricciones y ejemplos (in-context learning) que se inyecta como entrada en la ventana de contexto de un LLM para condicionar probabilísticamente su respuesta.

Representa el mecanismo formal mediante el cual se parametrizan las intenciones del usuario (por ejemplo: requerimientos de voltaje, ganancia o ancho de banda). Asimismo, el aislamiento estricto de prompts entre sub-módulos es fundamental para mitigar la atrición o saturación de la ventana de contexto al abordar circuitos multietapa complejos.

1.0.3. Modelos de lenguaje grande

Un modelo de lenguaje grande es un modelo estadístico, entrenado sobre grandes volúmenes de texto, que estima la probabilidad del siguiente elemento de una secuencia a partir de los anteriores. La generación procede de forma autorregresiva: el modelo produce un elemento, lo incorpora al contexto y repite el proceso, de modo que la salida se construye paso a paso [12]. Esta capacidad de generación permite que el modelo transforme una descripción en lenguaje natural en una estructura de datos o en código, que es precisamente lo que el sistema necesita en dos puntos: la extracción de los parámetros eléctricos a partir de la descripción del circuito y el cálculo de los valores de los componentes de cada sub-bloque. Internamente, el modelo procesa el texto como una secuencia de unidades llamadas tokens y opera sobre una ventana de contexto de tamaño acotado, que fija cuánta información puede considerar en una sola consulta; esta limitación reaparece más adelante como una de las razones para descomponer el circuito en sub-bloques.

Dos propiedades de estos modelos resultan pertinentes para el diseño. La primera es el aprendizaje en contexto: el modelo ajusta su respuesta a partir de los ejemplos e instrucciones incluidos en la propia consulta, sin necesidad de reentrenarlo [13]. Esto permite proporcionarle las ecuaciones de diseño de un sub-bloque como contexto y obtener los valores correspondientes. La segunda es la generación de salida estructurada, en ocasiones mediante mecanismos de llamada a funciones, que obliga al modelo a responder en un formato definido —por ejemplo, un objeto JSON con el tipo de circuito y sus parámetros— y facilita el procesamiento posterior de la respuesta.

La limitación que motiva el problema de investigación es complementaria a estas capacidades. El modelo genera texto plausible, pero no comprueba por sí mismo lo que produce [14]. En el dominio de los circuitos, esto se traduce en componentes inexistentes, restricciones físicas violadas y salidas eléctricamente inconsistentes que rara vez funcionan al primer intento [1], [4]. El modelo no dispone de un mecanismo interno que contraste su propuesta con el comportamiento físico del circuito; produce una respuesta que parece correcta según los patrones del lenguaje, no según las leyes de la electrónica. Esta brecha entre plausibilidad lingüística y corrección física es la que justifica incorporar una verificación externa al modelo, y es el punto del que parte el diseño del sistema: la simulación aporta la comprobación que el modelo no realiza.

La generación del modelo no es determinista. En cada paso, el siguiente elemento se elige de una distribución de probabilidad, y un parámetro de temperatura regula cuánta variación se admite en esa elección. Una temperatura baja produce salidas más estables y, al fijarla, hace repetible la generación, condición necesaria para comparar ejecuciones

durante la evaluación. Esta misma variabilidad sustenta la estrategia de referencia frente a la que se contrasta el sistema. El enfoque *best-of-N* consiste en pedir al modelo N propuestas independientes para la misma descripción, evaluarlas y conservar la mejor [15]. Es una forma directa de aprovechar el muestreo, pero no incorpora realimentación: cada propuesta se genera sin información sobre los fallos de las anteriores. El sistema propuesto se mide contra esta línea base a igual número de llamadas al modelo, lo que permite distinguir la mejora atribuible al lazo de ajuste de la que se obtendría por el solo hecho de muestrear más.

1.0.4. LangGraph

Es un marco de trabajo (framework) de ingeniería de software orientado al desarrollo de aplicaciones agenticas y flujos multiagente cíclicos, modelándolos formalmente como Grafos Dirigidos o Máquinas de Estados Finitas (FSM). En su arquitectura, los nodos representan agentes o funciones computacionales, los bordes definen las transiciones lógicas (incluyendo bifurcaciones condicionales), y el estado (State) constituye un bus de memoria estructurado e inmutable.

LangGraph coordina de manera declarativa la interacción entre los diferentes agentes del subsistema servidor. Permite ejecutar el patrón Map-Reduce en paralelo para sub-bloques funcionales y gestionar bordes condicionales que reinyectan diagnósticos y habilitan lazos de optimización iterativos en caso de fallo.

1.0.5. LangFuse

Es una plataforma de ingeniería y observabilidad de código abierto (LLM Observability and Tracing Platform) diseñada para rastrear, auditar y monitorizar el ciclo de vida de aplicaciones basadas en modelos de lenguaje. Facilita la telemetría detallada de trazas de ejecución (traces y spans), registro de latencias, consumo de tokens/costes asociados y evaluación cualitativa/cuantitativa de salidas y llamadas a herramientas (tool calling).

Proporciona la infraestructura de auditoría y trazabilidad en tiempo de ejecución. Permite inspeccionar qué instrucciones exactas recibió cada agente, monitorear el costo computacional de las llamadas concurrentes en la fase Map-Reduce y correlacionar los errores o reintentos de diseño con las métricas de respuesta de cada modelo.

1.0.6. Spice (Simulation Program with Integrated Circuit Emphasis)

Desarrollado originalmente en la Universidad de California en Berkeley, SPICE es el software estándar industrial y académico para la simulación numérica y análisis del comporta-

miento dinámico de circuitos electrónicos analógicos. Modela circuitos a partir de un listado de interconexiones (netlist) y resuelve numéricamente sistemas de ecuaciones algebraico-diferenciales no lineales empleando técnicas como el análisis nodal modificado y métodos iterativos de Newton-Raphson.

Constituye el entorno cuantitativo de validación física real (a través de motores como ngspice). Actúa como el criterio de verdad técnica indispensable para verificar parámetros como ganancia, margen de fase o rizo, y confirmar la ausencia de anomalías topológicas (nodos flotantes, cortocircuitos o fallos de convergencia).

1.0.7. PySpice

Es una biblioteca de abstracción e interfaz para Python que encapsula la sintaxis de simulación de circuitos electrónicos y establece un puente programático con motores de simulación SPICE (particularmente ngspice). Provee una interfaz orientada a objetos para declarar componentes, topologías y directivas de simulación (.AC, .DC, .TRAN, .OP) sin requerir el ensamblado manual de texto plano.

Es la herramienta empleada por el Agente de Escritura para traducir las estructuras de datos abstractas generadas por los agentes de cálculo a código de simulación estructurado y ejecutable. Asimismo, captura vectorialmente los resultados numéricos emitidos por el simulador para que el sistema evalúe el desempeño del circuito en bucle cerrado.

1.0.8. Generación Aumentada por Recuperación (RAG)

En esta arquitectura, la Generación Aumentada por Recuperación (RAG) sustituye los esquemas basados en ensayo y error o aprendizaje por refuerzo puro, transformando el subsistema en un entorno guiado por conocimiento empírico y documental indexado.

El RAG se estructura a través del Módulo Curador de Conocimiento y opera en tres dimensiones fundamentales:

Recuperación Previa y Anclaje Técnico (Pre-Generation Retrieval) Cuando el usuario describe un circuito en lenguaje natural, el sistema no permite que los LLMs asuman parámetros arbitrarios. El mecanismo de RAG realiza una búsqueda vectorial y semántica sobre bases de datos técnicas locales: Catálogos de Componentes COTS (Commercial Off-The-Shelf): Hojas de datos (datasheets) y tablas paramétricas de componentes reales del mercado (diodos rectificadores, capacitores de filtrado, transistores BJT/MOSFET y reguladores integrados). Plantillas de Topologías Analógicas Validadas: Estructuras de referencia netlist probadas para el tipo de circuito solicitado (filtros pasivos, fuentes lineales, configuraciones de amplificadores). Directivas de Simulación para SPICE: Parámetros de convergencia y tolerancias

(.OPTIONS, condiciones iniciales .IC). El Curador extrae estos fragmentos pertinentes y los inyecta en el prompt inicial del Master y de los Workers, restringiendo el espacio de diseño a parámetros realizables en hardware físico real.

B. Mitigación de Alucinaciones Topológicas y de Sizing Los modelos de propósito general tienden a inventar valores ideales (p. ej., resistores de valores no normalizados) o directivas incompatibles con los motores SPICE. La inyección dinámica de contexto mediante RAG actúa como un filtro (grounding): obliga al agente de cálculo a elegir exclusivamente modelos que cuenten con archivos de biblioteca (.lib / .model) reconocidos por el simulador ngspice.

Lazo Cerrado de Curación y Memoria de Diseño (Knowledge Curation Loop) La aportación diferencial del RAG en este sistema radica en su funcionamiento en lazo cerrado tras la ejecución física en ngspice:

Captura del Diagnóstico: Si ngspice reporta una divergencia numérica (fallo de Newton-Raphson, timestep too small) o una discrepancia severa respecto a la especificación esperada, el Agente Shell captura el log crudo. **Destilación Curada:** En lugar de reinyectar el mensaje de error textual íntegro al modelo generador, el Curador procesa el fallo, identifica la causa física subyacente y genera una directiva de ajuste o regla de diseño aprendida. **Consolidación en la Base Vectorial:** Esta heurística verificada se almacena en el almacén de conocimiento persistente (Self-Evolving Memory / Base Vectorial). En el reintento actual (o en futuros diseños de la misma familia de circuitos), el sistema recupera dicha directiva, impidiendo que el ecosistema caiga en ciclos repetitivos de error y acelerando la convergencia a una solución funcional.

1.0.9. Sistemas multiagente y orquestación

En tareas complejas de ingeniería analógica —como la síntesis, dimensionamiento y verificación de circuitos electrónicos—, el uso de un único Modelo de Lenguaje Grande (Single LLM Agent) genera un cuello de botella crítico: la atrición o saturación de la ventana de contexto (context attrition). A medida que se incrementa el número de directivas, restricciones topológicas y mensajes de depuración, el modelo tiende a alucinar nodos flotantes, interconexiones erráticas o componentes comerciales inexistentes. Un Sistema Multiagente (Multi-Agent System - MAS) resuelve esta limitación mediante el principio de descomposición funcional (divide y vencerás), desacoplando las responsabilidades en agentes autónomos con prompts pequeños, especializados y estrictamente aislados.

La orquestación centralizada no se implementa como un script secuencial rígido, sino como una Máquina de Estados Finita (FSM) estructurada como un grafo cíclico dirigido mediante LangGraph: Estado Compartido (State): Constituye el bus de datos inmutable y

central donde se registran la especificación formal del circuito, las variables intermedias, las listas de componentes calculadas y el historial de iteraciones. Nodos y Bordes Condicionales: Los nodos encapsulan a los agentes especializados, mientras que los bordes condicionales evalúan en tiempo de ejecución las transiciones del flujo, gestionando lazos cerrados de optimización (feedback loops) y detenciones de seguridad ante fallas críticas.

Para resolver circuitos analógicos multietapa (como fuentes reguladas CA/CD compuestas por etapas de transformación, rectificación, filtrado y regulación lineal), el sistema implementa el patrón concurrente Master-Worker estructurado en dos fases

Nodo Maestro (Master):

Recibe la descripción desglosada y el plan del circuito desde el Orquestador. Su rol consiste en particionar el problema global en especificaciones locales independientes para cada sub-bloque funcional y asegurar que ningún sub-bloque exceda los límites de tokens de la ventana de contexto permitida.

Sub-agentes Trabajadores (Workers):

Se instancian de manera concurrente. Cada Worker resuelve de forma exclusiva el dimensionamiento y el planteamiento analítico de su etapa asignada (cálculo de impedancias, selección de valores comerciales COTS, corrientes de polarización). Al no recibir el código ni los errores de las demás etapas, se elimina el consumo excesivo de memoria operativa del modelo.

1.0.10. Arquitectura de software: principios y patrones de coordinación

La forma del sistema se apoya en principios reconocidos de la ingeniería de software.

1.0.11. Separacion de intereses

cada parte del sistema atiende una preocupación distinta, de modo que las responsabilidades no se mezclan y cada una puede evolucionar por separado [16]. El proyecto aplica este principio al dividir el sistema en un subsistema cliente, que gestiona el acceso de los usuarios, y un subsistema servidor, que aloja la lógica de diseño de circuitos. La gestión de usuarios y la generación de circuitos son preocupaciones independientes y se despliegan por separado.

1.0.12. Ausencia de estado en el componente a escalar

Una instancia sin estado no guarda información entre solicitudes, de modo que cualquier instancia puede atender cualquier solicitud y el sistema admite replicar el servidor detrás

de un balanceador de cargas para repartir la demanda [17]. Este escalado horizontal es la razón por la que el subsistema servidor se diseña sin estado.

1.0.13. La ausencia de estado entre solicitudes convive con una exigencia opuesta dentro de cada solicitud

El lazo de ajuste necesita conservar el estado entre iteraciones, porque cada ciclo parte del resultado del anterior. El sistema resuelve esta tensión mediante un mecanismo de *checkpoint* que persiste el estado de la generación en una base de datos tras cada paso, de modo que el estado de una solicitud en proceso queda disponible para todos los agentes a lo largo de las iteraciones sin residir en la memoria de una instancia concreta. La coordinación entre agentes se modela como un grafo de estado, en el que los nodos son los agentes y las aristas el flujo de control, y sobre el que se apoya la persistencia por *checkpoint* [10]. Estos conceptos —separación de intereses, ausencia de estado, persistencia explícita y grafo de estado— son los que el capítulo de diseño aplica para justificar la estructura concreta del sistema.

1.1. El diseño de circuitos analógicos como proceso iterativo

El diseño de un circuito analógico no se resuelve con un procedimiento cerrado que, a partir de una especificación, devuelva una única respuesta correcta. El proyectista parte de un conjunto de metas eléctricas —una ganancia, una frecuencia de corte, un punto de polarización— y propone una topología y unos valores de componentes que, según las ecuaciones de diseño, deberían cumplirlas. La propuesta se verifica mediante simulación o medición, se comparan los resultados con las metas y, cuando hay desviación, se ajustan los valores y se repite el proceso. Este ciclo de proponer, simular, revisar y ajustar se recorre varias veces antes de que el circuito cumpla las metas [18].

Las ecuaciones de diseño son el vínculo entre el dominio de la especificación y el de la implementación. Para un amplificador no inversor con amplificador operacional, por ejemplo, la ganancia se relaciona con el cociente de dos resistencias; para un filtro RC de primer orden, la frecuencia de corte queda determinada por el producto de una resistencia y una capacitancia. Estas relaciones permiten calcular un valor inicial, pero no garantizan que el circuito real cumpla la meta, porque el modelo ideal ignora efectos que la simulación sí captura: la carga entre etapas, las no idealidades de los componentes y la interacción entre sub-bloques. De ahí que la simulación funcione como árbitro: es el mecanismo que comprueba si los valores calculados producen el comportamiento esperado.

El proyecto se restringe a circuitos analógicos de corriente alterna y corriente directa contruidos con componentes comerciales —resistores, capacitores, inductores, fuentes, diodos, transistores y amplificadores operacionales descritos por macromodelo—, en contraste con la línea de trabajo que diseña a nivel de transistor para un proceso de fabricación específico [19]. Esta distinción no es accesoria. El diseño a nivel de componente comercial trabaja con dispositivos cuyos modelos ya existen y cuyos valores se eligen de catálogos discretos, mientras que el diseño a nivel de transistor dimensiona dispositivos sobre un nodo tecnológico. La frontera entre ambos niveles delimita el alcance del proyecto y, como se argumenta en el estado del arte, marca el espacio que la propuesta ocupa.

1.2. Síntesis del marco teórico

Los conceptos expuestos forman una cadena que recorre el sistema de extremo a extremo. El dominio del diseño analógico fija el problema y explica por qué la simulación debe actuar como árbitro. El modelo de lenguaje aporta la capacidad de pasar del lenguaje natural a una propuesta de circuito, y su incapacidad de autoverificarse motiva el resto del diseño. La organización multiagente reparte ese trabajo en responsabilidades acotadas y, frente a la generación repetida sin realimentación, sustituye el muestreo ciego por un ciclo informado por la simulación. El aprendizaje por refuerzo da marco formal a la decisión que convierte las métricas de la simulación en una acción sobre el circuito. La simulación SPICE produce esas métricas, que son la señal objetiva que cierra el lazo. Y los principios de arquitectura sostienen el conjunto como un sistema que separa preocupaciones, escala y conserva el estado donde debe conservarlo. La articulación de estos elementos dentro de un único lazo de generación y validación, a nivel de componentes comerciales, es lo que el capítulo de diseño desarrolla.

Capítulo 2

Diseño y desarrollo del proyecto

2.1. Propuesta de Diseño

En el capítulo anterior se revisó el estado del arte de los sistemas multiagente basados en LLM para el diseño de circuitos analógicos, y se identificó que los trabajos reportados operan a nivel de transistor sobre un nodo tecnológico (PDK), mientras que la generación y validación de circuitos a nivel de componentes comerciales (COTS) con simulación SPICE dentro del lazo de diseño permanece como una brecha abierta. Este capítulo presenta la propuesta de solución que atiende esa brecha.

El propósito del capítulo es exponer el diseño del sistema antes de su construcción: las premisas que lo fundamentan, su descomposición funcional, su arquitectura estructural y su comportamiento en ejecución. Las decisiones de diseño se justifican a partir de la naturaleza del problema, de los principios de la ingeniería de software y de los hallazgos del estado del arte; los artefactos de análisis y diseño elaborados a lo largo del proyecto —la Especificación de Requisitos de Software (SRS), el modelo de casos de uso, el diagrama funcional, el modelo arquitectónico C4 de tres niveles y el diagrama de secuencia— registran y representan esas decisiones, de modo que cada una quede expresada en un artefacto verificable.

La exposición sigue un orden de lo abstracto a lo concreto. Primero se establecen el problema que el diseño debe resolver, las premisas que lo orientan y el papel del SRS como registro de los requisitos (sección 2.2). A continuación se describe *qué* hace el sistema mediante su descomposición funcional y sus casos de uso (sección 2.3). Después se describe *cómo* se estructura, mediante el modelo C4 en sus tres niveles (sección 2.4). Enseguida se describe *cómo se comporta* en ejecución, mediante el diagrama de secuencia del flujo principal de generación (sección 2.5). Finalmente se explican los atributos de calidad y seguridad que se desprenden del diseño (secciones 2.6 y 2.7), se enuncian los criterios de aceptación que la propuesta deberá satisfacer (sección 2.8) y se cierra con una síntesis del diseño y su conexión con la evaluación del sistema (sección 2.9).

2.2. Premisas y fundamento de diseño

2.2.1. El problema de diseño y el papel del SRS

El diseño responde a un problema concreto: transformar la descripción de un circuito analógico CA/CD, expresada en lenguaje natural, en un netlist que haya sido simulado y validado, empleando componentes comerciales en lugar de dispositivos a nivel de transistor. Este problema impone por sí mismo varias exigencias al diseño —la interpretación de lenguaje natural, el cálculo de valores de componentes, la simulación física y el ajuste del circuito hasta cumplir una meta— que son las que, en última instancia, justifican la forma del sistema. Las decisiones de diseño de este capítulo se fundamentan en esas exigencias del problema, en los principios de la ingeniería de software y en los hallazgos del estado del arte, y no en la conformidad con un documento interno.

La Especificación de Requisitos de Software (SRS) en su versión 1.0 cumple un papel distinto al de fuente de justificación: es el artefacto que registra de forma ordenada y verificable los requisitos que el sistema debe satisfacer, los traduce a criterios de verificación y delimita el alcance. El SRS fija qué se construye —el alcance, los requisitos funcionales por subsistema, los requisitos no funcionales y los criterios de aceptación—; las razones por las que el diseño adopta una u otra forma se exponen en este capítulo a partir del problema y de la disciplina, y los identificadores de requisito (RF, RNF) se citan únicamente para señalar qué requisito queda cubierto por cada decisión, no para sustentarla.

El alcance del sistema comprende recibir la descripción de un circuito analógico en lenguaje natural, dividir circuitos de varias etapas en sub-bloques, calcular los valores de los componentes, generar el código PySpice y el netlist, simular en ngspice, ajustar iterativamente los valores mediante un agente curador, y entregar al usuario el netlist final junto con las métricas y el historial de iteraciones. Quedan fuera del alcance el diseño de *layout* físico (PCB), la síntesis a nivel de transistor para un nodo tecnológico y la generación de esquemáticos gráficos.

2.2.2. Premisas de diseño

A partir del problema descrito, y guiado por principios reconocidos de la ingeniería de software, el diseño adopta un conjunto de premisas que orientan las decisiones arquitectónicas de las secciones siguientes. Para cada una se expone primero su fundamento y, después, el requisito del SRS que queda cubierto.

La primera premisa es la separación del sistema en dos subsistemas que se despliegan por separado. Esta decisión aplica el principio de separación de intereses: las responsabilidades

de acceso —autenticación, sesiones, espacios de trabajo y emisión de API keys— pertenecen a un dominio distinto del de la generación de circuitos, y mantenerlas en subsistemas separados reduce el acoplamiento entre ambos y permite desplegarlos, escalarlos y razonar sobre ellos de forma independiente. De ahí que el subsistema cliente actúe como puerta de entrada de los usuarios y no ejecute lógica de diseño de circuitos, que reside íntegramente en el subsistema servidor.

Dentro del servidor, el diseño se rige por la modularidad de los agentes, que materializa los principios de alta cohesión y bajo acoplamiento: cada agente concentra una responsabilidad única y se comunica con los demás solo a través de un estado compartido, sin conocer su implementación interna. Esta organización permite reemplazar o modificar un agente sin alterar los restantes, propiedad que la literatura sobre sistemas multiagente para diseño de circuitos asocia con la capacidad de evolucionar y depurar cada agente por separado, y que en el SRS queda recogida como requisito de mantenibilidad (RNF-04.1).

A esta modularidad se suma la concurrencia en el cálculo, que no es una decisión arbitraria sino una consecuencia de la estructura del problema: un circuito de varias etapas se descompone en sub-bloques, y aquellos que no dependen entre sí pueden calcularse de forma simultánea sin afectar el resultado. El diseño aprovecha esta independencia mediante un patrón Master-Worker con *Map-Reduce*, en el que un nodo Master reparte cada sub-bloque independiente como tarea a un worker y luego recombina los resultados. Esta correspondencia entre la naturaleza paralelizable del problema y la estructura concurrente del sistema queda cubierta por los requisitos de reparto y desempeño del SRS (RF-01.3, RNF-01.2).

El rasgo que distingue la propuesta frente a los enfoques revisados en el estado del arte es la validación dentro del lazo de diseño. Los sistemas multiagente reportados para diseño analógico tienden a emplear la simulación como comprobación posterior a la generación; la propuesta, en cambio, la incorpora como parte del proceso de ajuste: el agente curador compara las métricas obtenidas de ngspice con las metas y decide si el circuito se acepta o se vuelve a ajustar, hasta cumplir la meta o alcanzar un número máximo de iteraciones. Esta integración de la simulación en el lazo es la traducción, al nivel de componentes comerciales, del cierre de realimentación que la literatura sobre diseño asistido por LLM ha mostrado eficaz al nivel de transistor, y constituye la aportación central de la propuesta; en el SRS se recoge como el control del ciclo de ajuste (RF-01.4).

Sostener ese lazo exige conservar el estado entre iteraciones, por lo que el diseño conserva de forma explícita el estado de la generación mediante un mecanismo de *checkpoint* sobre una base de datos, de modo que una solicitud en proceso mantenga su estado disponible para todos los agentes a lo largo de las iteraciones. Esta persistencia se modela de manera explícita en el comportamiento dinámico del sistema (sección 2.5) por ser una

propiedad esencial del lazo, y no una nota accesoria. Finalmente, y en línea con el principio de configurabilidad sin recompilación, los parámetros del agente curador —los pesos, factores y umbrales del cálculo de recompensa, junto con el número máximo de iteraciones— se mantienen en un archivo de configuración externo, de modo que puedan ajustarse durante la experimentación sin modificar el código (RNF-04.2).

2.3. Descomposición funcional y casos de uso

2.3.1. Vista funcional

El sistema ofrece un conjunto acotado de funciones que se desprenden directamente del problema y quedan registradas en el SRS: dar acceso a los usuarios y gestionar sus sesiones, espacios de trabajo y API keys (cliente); leer una descripción de circuito en lenguaje natural y extraer de ella los parámetros eléctricos; dividir un circuito de varias etapas en sub-bloques; calcular los valores de los componentes de cada sub-bloque; generar el código PySpice y el netlist; simular el netlist en ngspice y leer las métricas; comparar las métricas con las metas y decidir si el circuito se acepta o se ajusta; y entregar al usuario el netlist final junto con las métricas y el historial de iteraciones.

El diagrama funcional organiza estas funciones según el subsistema responsable y el flujo de datos que las relaciona, lo que permite ver la propuesta como un encadenamiento de transformaciones sobre la descripción del circuito, desde el lenguaje natural hasta el netlist validado.

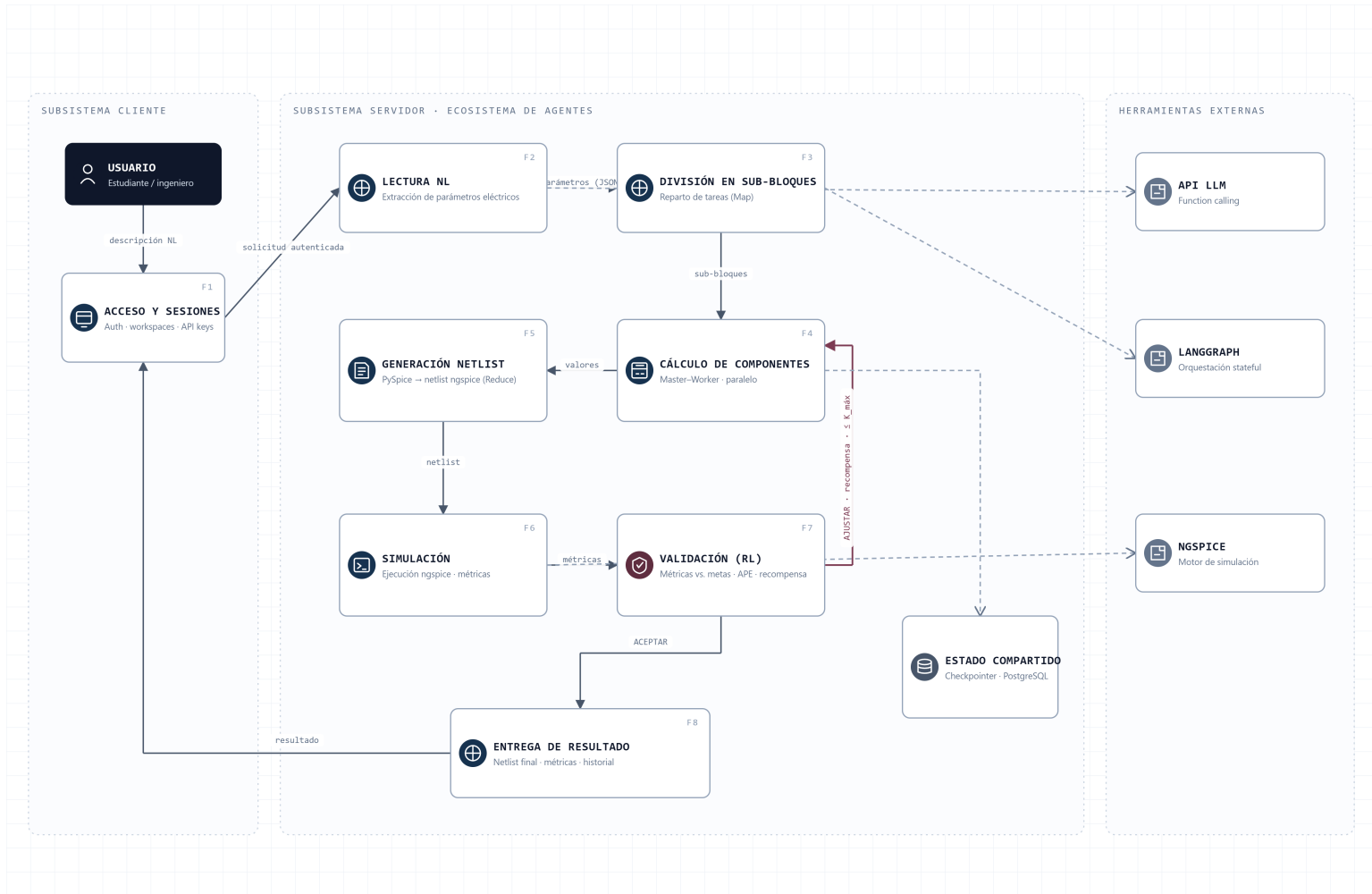


Figura 2.1. Diagrama funcional del sistema.

2.3.2. Modelo de casos de uso

Los casos de uso describen las interacciones que los actores ejecutan sobre el sistema. Se identifican dos clases de actor: el **usuario**, que interactúa con el subsistema cliente, y los **servicios externos** (la API del LLM y el motor ngspice), que el subsistema servidor consume durante la generación.

Los casos de uso principales del usuario incluyen: registrarse e iniciar sesión, gestionar espacios de trabajo, emitir y revocar API keys, y enviar una solicitud de generación de circuito junto con su descripción en lenguaje natural. El caso de uso central —*generar circuito*— desencadena el flujo del subsistema servidor, que se detalla en su comportamiento dinámico en (sección 2.5). Los diagramas de casos de uso restantes (acceso, generación y decisión) se incluyen en el apéndice A.

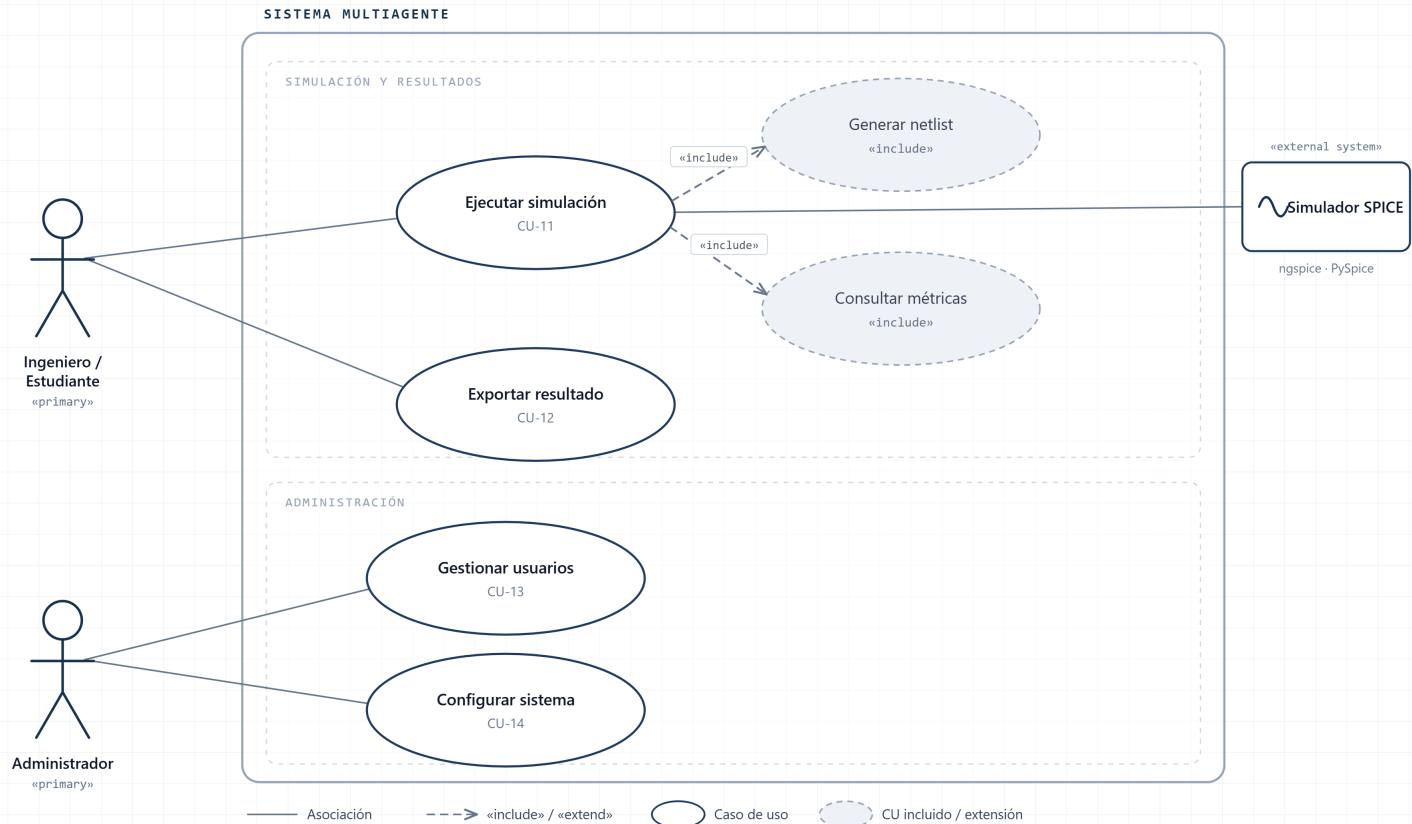


Figura 2.2. Diagrama de casos de uso: flujo de simulación y validación.

2.4. Arquitectura del sistema (modelo C4)

2.4.1. Nivel 1 — Contexto

El nivel de contexto sitúa el sistema frente a sus actores y sistemas externos. Muestra al usuario interactuando con el sistema, y al sistema consumiendo dos servicios externos: la API del LLM y el motor de simulación ngspice. Este nivel deja claro que el sistema es una unidad que media entre la intención del usuario, expresada en lenguaje natural, y dos capacidades externas: el razonamiento del LLM y la simulación física del circuito.

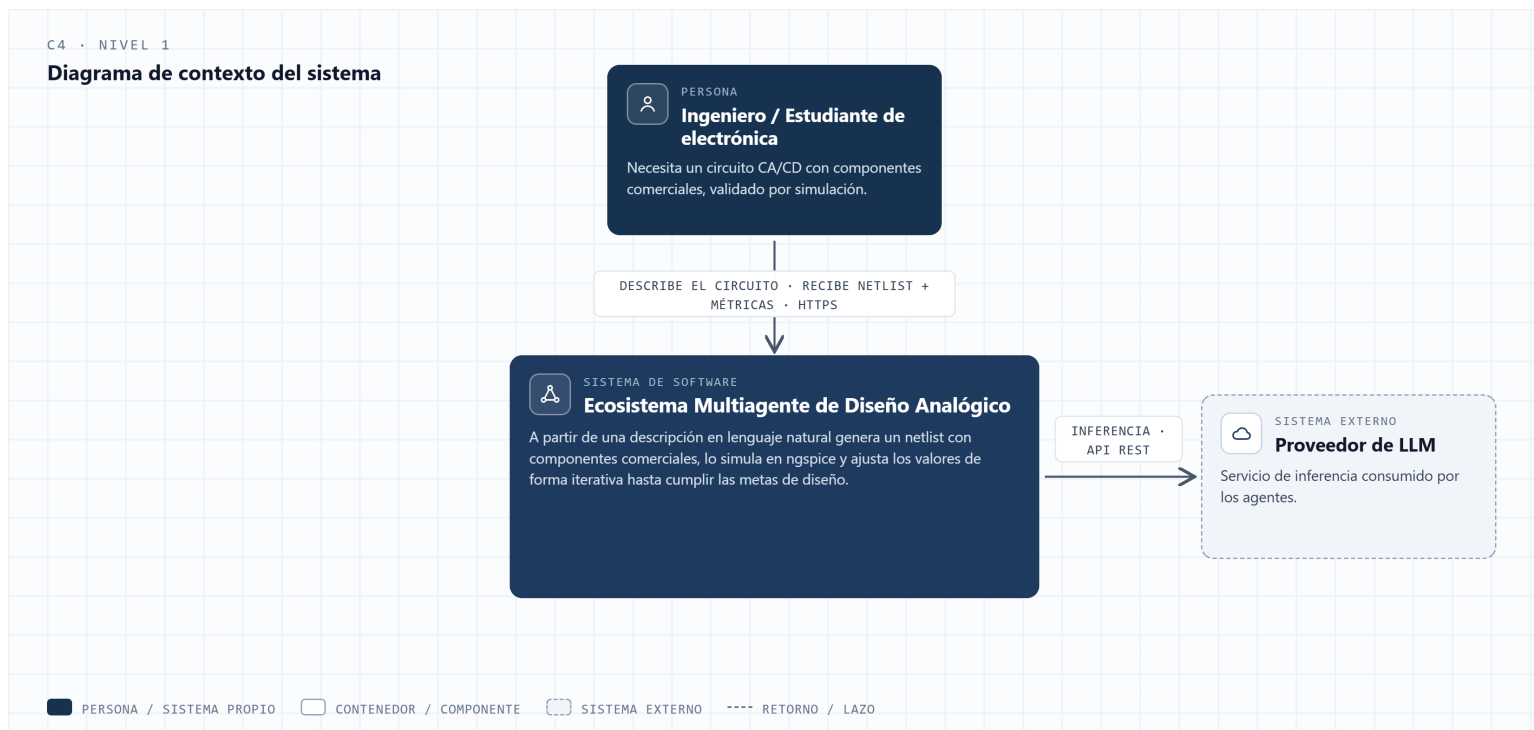


Figura 2.3. C4 Nivel 1: Diagrama de contexto.

2.4.2. Nivel 2 — Contenedores

El nivel de contenedores descompone el sistema en sus unidades desplegables. Hace explícita la separación cliente/servidor: el subsistema cliente (autenticación, sesiones, espacios de trabajo, API keys) y el subsistema servidor (el ecosistema de agentes), junto con la base de datos de persistencia y la frontera con los servicios externos. Este nivel justifica la premisa de despliegue independiente (sección 2.2.2) y muestra el camino de una solicitud autenticada desde el cliente hacia el servidor.

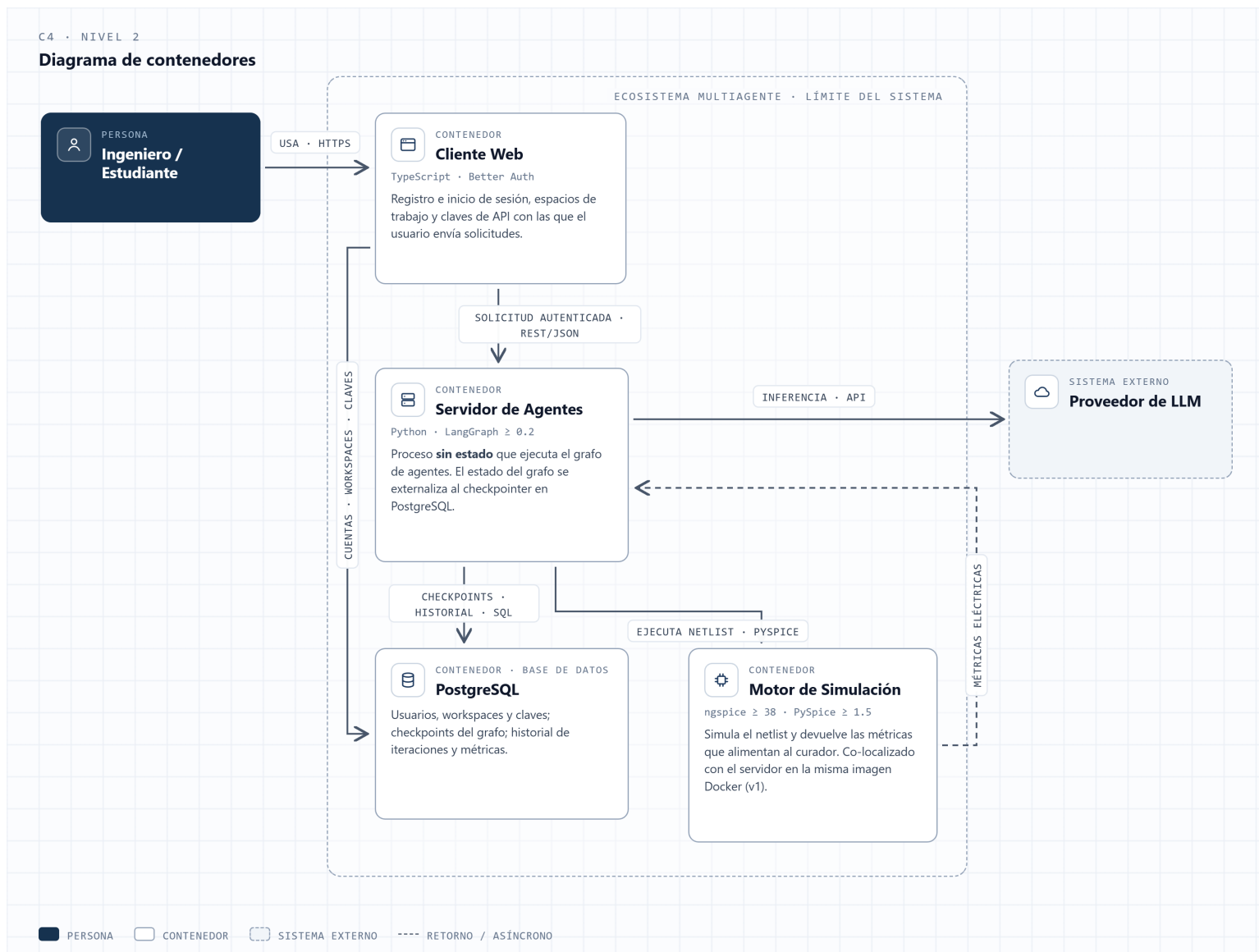


Figura 2.4. C4 Nivel 2: Diagrama de contenedores.

2.4.3. Nivel 3 — Componentes

El nivel de componentes abre el subsistema servidor y muestra el ecosistema de agentes: el agente Orquestador (entrada del servidor y coordinador del estado compartido), el agente de Cálculo (Master-Worker), el agente Curador (RL, lazo de validación iterativa), el agente de Shell (ejecución de ngspice) y el agente de Escritura (generación del netlist con PySpice), todos coordinados sobre el grafo de orquestación con persistencia en base de datos. Este nivel materializa la premisa de modularidad de agentes (sección 2.2.2) y prepara el terreno para la descripción del comportamiento dinámico.

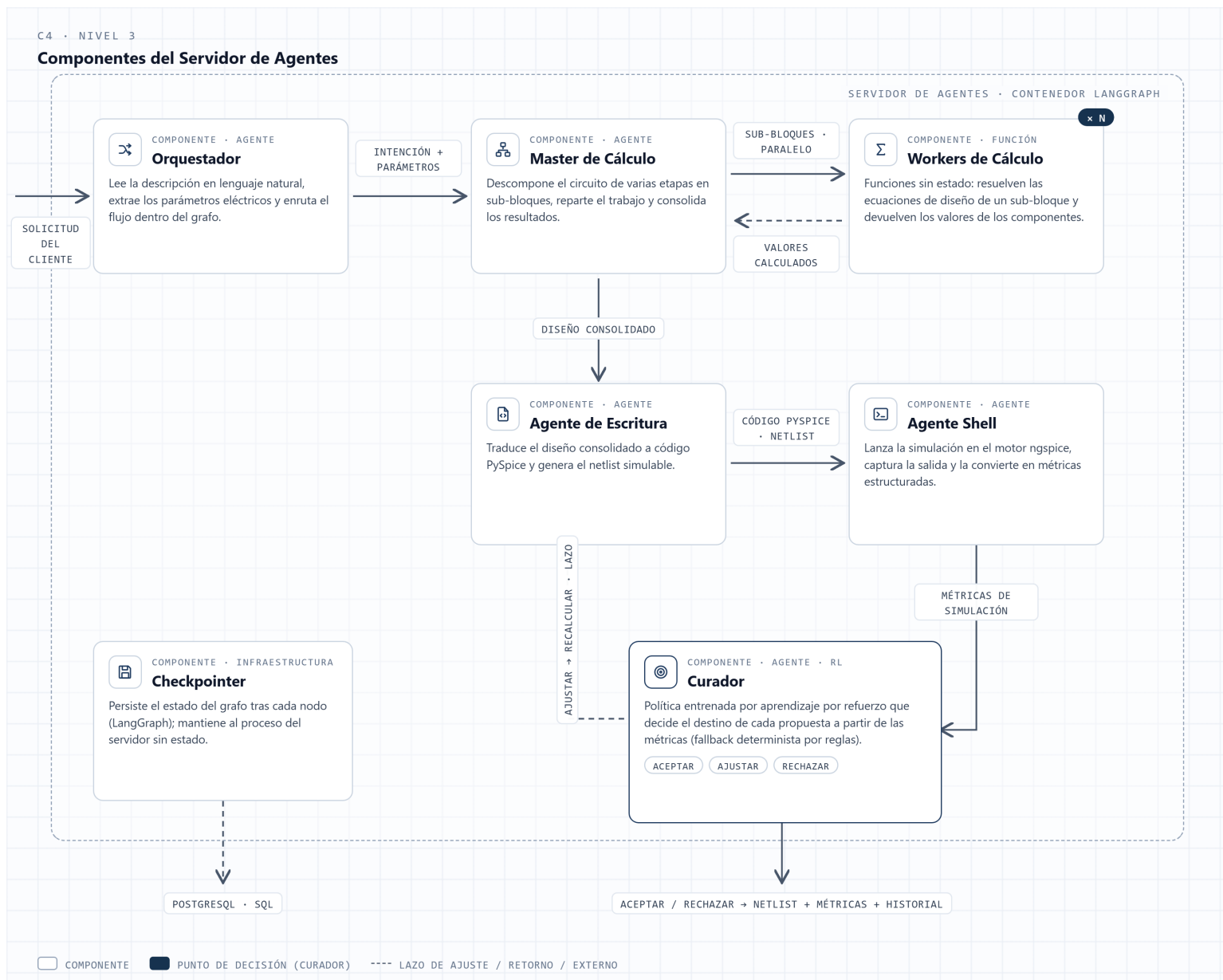


Figura 2.5. C4 Nivel 3: Diagrama de componentes.

2.5. Comportamiento dinámico: flujo de generación

Las secciones anteriores describen la estructura estática del sistema. Esta sección describe su comportamiento en ejecución: cómo colaboran los agentes en el tiempo para transformar una descripción en lenguaje natural en un netlist validado. El diagrama de secuencia integra las vistas anteriores en una línea temporal.

El flujo principal procede así: el orquestador recibe la solicitud autenticada y la descripción del circuito, usa el LLM para extraer los parámetros y la lista de sub-bloques,

y reparte los sub-bloques independientes en paralelo al agente de cálculo (Master-Worker). El agente de escritura genera el código PySpice y el netlist; el agente de shell lo simula en ngspice y lee las métricas; el agente curador compara las métricas con las metas y decide. Según su decisión, el ciclo se repite —ajustando los valores— o termina. El ciclo está acotado por un número máximo de iteraciones configurable (parámetro de guarda del lazo).

El comportamiento de persistencia se modela de forma explícita: el estado de cada iteración se conserva mediante el *checkpoint*, lo que se representa como un fragmento de referencia en el diagrama, de modo que la naturaleza con estado del sistema quede formalmente reflejada y no como una observación accesorio.

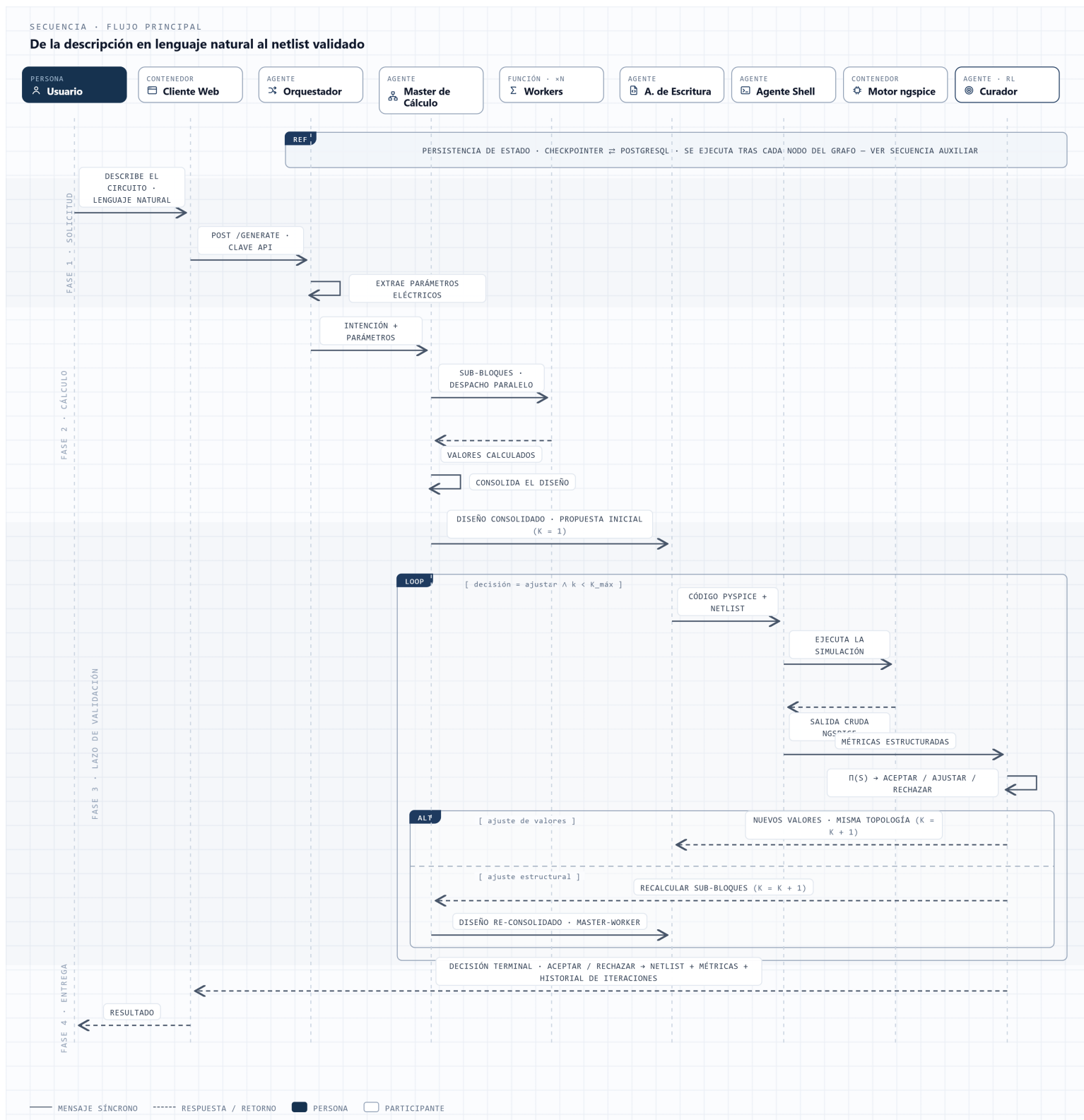


Figura 2.6. Diagrama de secuencia del flujo de generación.

2.6. Calidad de la solución

La calidad de la solución se entiende aquí como el grado en que el sistema satisface las necesidades para las que se diseña, evaluado sobre características observables y no sobre el juicio del diseñador. El modelo de calidad del producto de la norma ISO/IEC 25010 organiza esas características y ofrece un vocabulario común para nombrarlas [20]. Se adopta el modelo de calidad del producto —y no el de calidad en uso, centrado en la experiencia del usuario final— porque el sistema se evalúa como un artefacto técnico operado mediante una interfaz de programación. De las características que define la norma, el diseño atiende de forma directa cinco: idoneidad funcional, fiabilidad, eficiencia de desempeño, mantenibilidad y seguridad. A ellas se suma la trazabilidad de la generación, que el SRS recoge como requisito no funcional. Cada atributo se apoya en una decisión de diseño ya expuesta y su comprobación se registra como criterio de aceptación o como requisito no funcional, cuya medición corresponde al capítulo de pruebas.

La idoneidad funcional expresa si el sistema hace lo que debe: generar, a partir de una descripción en lenguaje natural, un circuito analógico CA/CD cuyos parámetros se acercan a las metas. Esta característica es la razón de ser del sistema y se concreta en métricas que la hacen medible —la tasa de resolución y el error porcentual absoluto medio—, contrastadas además frente a una línea base *best-of-N*. El lazo de ajuste guiado por el curador es el mecanismo que la persigue: itera hasta acercar las métricas a las metas o hasta agotar el número de iteraciones.

La fiabilidad se refiere a que el sistema se comporte de forma estable ante las condiciones esperadas y ante el fallo. El diseño la atiende por tres vías. El lazo de ajuste está acotado por un número máximo de iteraciones, lo que impide que el proceso se prolongue de forma indefinida ante un circuito que no converge. El curador dispone de una decisión determinista por reglas como respaldo de su política, de modo que el sistema produce siempre una decisión aunque la política aprendida no resulte aplicable. Y el *checkpoint* aporta recuperabilidad: al persistir el estado de la generación tras cada paso, una ejecución interrumpida puede retomarse desde el último punto guardado en lugar de reiniciarse, propiedad que el SRS recoge como tolerancia a fallos (RNF-03.2). La convergencia de la simulación se registra como una métrica y forma parte del estado que el curador evalúa, lo que evita que un netlist no convergente se acepte como válido.

La eficiencia de desempeño atañe al tiempo y a los recursos que el sistema consume para producir un resultado. El diseño la aborda mediante el cálculo concurrente de los subbloques independientes, que reduce la latencia frente a un cálculo secuencial, y mediante un límite de tiempo para el ciclo completo. El carácter sin estado del subsistema servidor habilita

además su réplica detrás de un balanceador de cargas, lo que permite atender más solicitudes en paralelo sin rediseñar el sistema; esta capacidad de crecer mediante réplicas es lo que el SRS registra como escalabilidad y que el modelo de la norma sitúa dentro de la eficiencia de desempeño.

La mantenibilidad se fundamentó ya entre las premisas de diseño (sección 2.2.2): la modularidad de los agentes materializa la alta cohesión y el bajo acoplamiento, y la configuración externa de los pesos, los factores y los umbrales del curador permite ajustar el comportamiento del sistema sin tocar el código. Se retoma aquí para situarla dentro del modelo de calidad, no para repetir su justificación.

A la mantenibilidad se vincula la trazabilidad de la generación. El sistema conserva el historial de iteraciones y la recompensa que el curador calcula en cada ciclo, de modo que el camino que llevó a un netlist queda registrado y es posible reconstruirlo. En términos del modelo de la norma, esta propiedad se aproxima a la analizabilidad, una faceta de la mantenibilidad, pues facilita diagnosticar el comportamiento del sistema tras una ejecución. El mismo registro constituye la traza en la que se apoya el control de seguridad descrito en la sección 2.7, lo que enlaza la calidad y la seguridad sobre un único mecanismo.

La seguridad es, en el modelo ISO/IEC 25010, una característica de calidad más. Por su peso en un sistema que gestiona credenciales y claves de acceso, se trata de forma separada en la sección siguiente.

La correspondencia entre cada característica, la decisión de diseño que la atiende y el lugar donde se registra o se mide se resume a continuación.

Tabla 2.1. Correspondencia entre atributos de calidad, decisiones de diseño y evidencia de evaluación.

Característica	Decisión de diseño que la atiende	Dónde se registra o mide
Idoneidad funcional	Lazo de ajuste guiado por el curador	Tasa de resolución y MAPE; comparación con <i>best-of-N</i>
Fiabilidad	Límite de iteraciones; respaldo determinista del curador; recuperabilidad por <i>checkpoint</i> ; registro de convergencia	CA-04, CA-05; RNF-03.2
Eficiencia de desempeño	Cálculo concurrente; límite de tiempo; servidor sin estado replicable	RNF-01.1, RNF-01.2; CA-05, CA-06
Mantenibilidad	Modularidad de los agentes; configuración externa	RNF-04.1
Trazabilidad (anali- zabilidad)	Registro del historial de iteraciones y de la recompensa	CA-03; RNF de trazabilidad
Seguridad	Controles descritos en la sección 2.7	RF-06, RIMP-11

Conviene precisar el alcance de esta evaluación de calidad. La idoneidad funcional, la fiabilidad y la eficiencia de desempeño se contrastan contra umbrales numéricos en el capítulo de pruebas. La mantenibilidad, la trazabilidad y la seguridad, en cambio, se evidencian de forma estructural —por las propiedades del diseño y por la revisión del código y del repositorio— y no mediante una métrica cuantitativa, lo que constituye un límite reconocido de la evaluación. Quedan fuera del foco de calidad de esta versión la usabilidad —el sistema se opera mediante una interfaz de programación, no mediante una interfaz gráfica para usuarios finales— y la portabilidad más allá del despliegue reproducible en contenedores.

2.7. Seguridad de la solución

La seguridad del sistema se aborda en proporción a su naturaleza de prototipo académico: el objetivo no es una auditoría completa, sino controles razonables sobre los activos que el sistema maneja. Esos activos son las credenciales de los usuarios, las API keys con las que se autentican las solicitudes, los secretos del propio sistema —la clave de la API del LLM, las

credenciales de la base de datos y los secretos de Better Auth—, las descripciones y netlists de cada usuario y la integridad del estado que el servidor persiste durante la generación. El análisis se ordena con los tres principios clásicos de la seguridad informática: confidencialidad, integridad y disponibilidad [21].

Bajo confidencialidad se agrupan los secretos y las credenciales, que no deben quedar expuestos. A esta categoría pertenece también la descripción del circuito que el usuario envía, que el sistema transmite a un proveedor de LLM externo para su procesamiento; la confidencialidad de esos datos depende en parte de ese proveedor, dependencia que se reconoce como límite al final de la sección. La integridad se refiere a que el estado de una generación y el netlist resultante no se alteren de forma indebida entre iteraciones. La disponibilidad —que el servicio responda a las solicitudes legítimas— se apoya en el diseño sin estado del servidor, que admite réplicas y reduce el impacto de la caída de una instancia.

El diseño incorpora un conjunto de controles proporcionados a esos activos. La autenticación de los usuarios se delega en Better Auth [22], que concentra el registro, el inicio de sesión y la gestión de sesiones en un componente especializado en lugar de implementarlos desde cero. La autorización de las solicitudes al servidor se realiza mediante API keys que el usuario emite y puede revocar, de modo que cada solicitud que llega al servidor viaja autenticada. Las API keys y los circuitos de cada usuario quedan acotados a su espacio de trabajo, por lo que la autorización no se limita a comprobar que una solicitud esté autenticada, sino a que cada usuario acceda solo a sus propios recursos. La separación entre subsistema cliente y subsistema servidor actúa como frontera de confianza: el servidor no expone la lógica de diseño a solicitudes no autenticadas y el cliente no ejecuta esa lógica. Los secretos del sistema se leen de variables de entorno y no se escriben en el código ni se versionan en el repositorio, control que el SRS recoge como requisito de manejo de secretos (RIMP-11).

Un riesgo propio de un sistema que recibe lenguaje natural y lo entrega a agentes basados en LLM es que la entrada del usuario contenga instrucciones que desvíen el comportamiento de los agentes. Para acotarlo, la descripción se valida y se restringe a los parámetros eléctricos que el orquestador necesita antes de incorporarla al proceso, y la salida del modelo se exige en un formato estructurado, lo que reduce el margen para respuestas fuera de lo previsto.

Un control específico de este sistema atiende la ejecución de artefactos producidos por el modelo. El agente de escritura genera código PySpice y un netlist, y el agente de shell los ejecuta para obtener las métricas; ejecutar código generado por un LLM es una superficie que conviene contener. El diseño la acota al ejecutar esos artefactos dentro del entorno contenedorizado del servidor, cuyo despliegue reproducible recoge el SRS (RIMP-10), de modo que la ejecución queda aislada del entorno anfitrión. El endurecimiento de

ese aislamiento más allá de la contención por contenedor queda fuera del alcance de esta versión. El registro de la actividad —entre otros, la recompensa que el curador calcula en cada iteración— deja una traza que permite revisar el comportamiento del sistema tras una ejecución.

Los controles anteriores se corresponden con varias de las categorías de riesgo que recopila el proyecto OWASP en su lista de los diez riesgos más críticos para aplicaciones web [23]. El control de acceso por API key y el aislamiento entre espacios de trabajo atienden el riesgo de control de acceso roto; la delegación en Better Auth, el de fallos de identificación y autenticación; el manejo de secretos fuera del repositorio, el de configuración de seguridad deficiente y exposición de datos sensibles; la validación de la entrada, el de inyección; y la contención de la ejecución de artefactos generados, el de fallos de integridad del software y de los datos. La lista de OWASP se emplea como catálogo de referencia para situar los controles, no como una verificación exhaustiva de conformidad.

El alcance de seguridad de esta versión tiene límites explícitos. La comunicación entre el usuario y el sistema se realiza sobre HTTPS, pero el diseño no incorpora cifrado de los datos en reposo más allá del que provea la base de datos, ni monitoreo de seguridad especializado, ni pruebas de penetración formales, ni un modelado de amenazas exhaustivo. A ello se añaden dos límites particulares: el sistema no aplica un control de tasa sobre las solicitudes, por lo que el uso intensivo de un usuario autenticado puede consumir recursos de cómputo y llamadas al LLM sin freno; y la confidencialidad de los datos enviados al proveedor de LLM externo escapa al control directo del sistema. Todos ellos exceden el alcance de un prototipo de tesina y se señalan para delimitar con honestidad qué protege el sistema y qué no.

2.8. Criterios de aceptación de la propuesta

La propuesta de solución se considerará satisfactoria si el sistema construido cumple un conjunto de criterios de aceptación objetivos y comprobables de forma independiente. Estos criterios están enunciados en el SRS junto con su procedimiento de verificación, lo que permite contrastarlos con mediciones reales en lugar de con el juicio del diseñador. No se evalúan en este capítulo —su comprobación empírica corresponde al capítulo de experimentos y resultados— pero se enuncian aquí como cierre del diseño, para dejar explícito qué deberá medirse y bajo qué condición se dará por cumplido.

Entre los criterios relevantes figuran: que el sistema acepte una descripción en lenguaje natural y extraiga de ella correctamente los parámetros eléctricos; que ejecute sub-bloques independientes en paralelo; que el lazo de ajuste itere hasta cumplir la meta o alcanzar el máximo de iteraciones; que produzca un netlist simulado en ngspice; y que la salida sea

repetible con la temperatura del LLM fijada, para permitir la comparación de resultados durante la evaluación.

2.9. Síntesis del diseño

La propuesta de solución descrita en este capítulo da forma concreta a la brecha identificada en el estado del arte: un sistema que genera y valida circuitos analógicos CA/CD a nivel de componentes comerciales, con la simulación SPICE incorporada dentro del lazo de diseño. El diseño se articula en tres planos que se sostienen entre sí. La descomposición funcional y los casos de uso (sección 2.3) establecen qué hace el sistema y quién interactúa con él. El modelo C4 en sus tres niveles (sección 2.4) establece cómo se estructura: la separación en subsistema cliente y subsistema servidor, y la organización del servidor como un ecosistema de agentes modulares —Orquestador, Cálculo, Curador, Shell y Escritura— coordinados sobre un grafo con estado y persistencia. El diagrama de secuencia (sección 2.5) establece cómo se comportan esos elementos en el tiempo, integrando el lazo de ajuste iterativo que distingue a la propuesta.

Los criterios de aceptación enunciados en (sección 2.8) fijan la condición que el sistema construido deberá cumplir. Su comprobación —la medición del comportamiento del sistema frente a esos criterios sobre un conjunto de circuitos de prueba— constituye la evaluación del sistema y se desarrolla en el capítulo siguiente, que toma este diseño como objeto de medida.

2.10. Introducción

El capítulo anterior expuso el diseño del sistema: su arquitectura en el modelo C4, su descomposición funcional y de casos de uso, su comportamiento dinámico, y los marcos de calidad (sección 2.6) y de seguridad (sección 2.7) que orientan la construcción. Este capítulo presenta la materialización de ese diseño en un prototipo funcional: el código fuente que lo implementa y su organización en el repositorio, las pantallas del prototipo en operación, y el análisis de seguridad de la implementación, que contrasta los controles previstos en el diseño con los efectivamente construidos.

2.11. Código fuente del proyecto

El código fuente completo del sistema se encuentra publicado en un repositorio de GitHub, lo que permite su revisión, su reproducción y el seguimiento de su historial de desarrollo:

<https://github.com/antoniotorresxd/ia-spice>

El repositorio se organiza como un monorepositorio con tres aplicaciones independientes bajo el directorio `project/apps/`, en correspondencia directa con los contenedores del nivel 2 del modelo C4 (sección 2.4.2):

- `client/` — el subsistema cliente: una aplicación web construida con React y TypeScript sobre Vite, que concentra la autenticación, los espacios de trabajo y la interfaz desde la que el usuario envía sus solicitudes en lenguaje natural.
- `server/` — la API del sistema: un servicio construido con el framework Hono sobre el runtime Bun, con Better Auth para la autenticación, Drizzle ORM sobre PostgreSQL (Neon) para la persistencia, y validación de esquemas con Zod. Expone, además de las rutas de autenticación, el catálogo de configuraciones de LLM que consume el subsistema de agentes.
- `agents/` — el ecosistema de agentes: un proyecto en Python 3.12 que implementa, sobre LangGraph, el grafo de orquestación descrito en el capítulo anterior —orquestador, cálculo (Master-Worker), síntesis (escritura y shell) y curador— con PySpice para la generación de netlists y el motor ngspice para la simulación.

Cada aplicación gestiona sus propias dependencias con su gestor correspondiente (npm, bun y uv, respectivamente) y cuenta con su propia suite de pruebas automatizadas, cuyos resultados se documentan en el capítulo de pruebas. El historial de *commits* del repositorio registra la evolución incremental del desarrollo, con mensajes bajo la convención *conventional commits* que distinguen funcionalidades, correcciones, pruebas y documentación.

2.12. Pantallas del prototipo funcional

Las siguientes capturas muestran el prototipo funcional en operación y evidencian que el subsistema cliente materializa los casos de uso de acceso y de solicitud de generación descritos en la sección 2.3.2.

La pantalla de inicio de sesión (figura 2.7) es la puerta de entrada al sistema. En su panel derecho ofrece el formulario de autenticación por correo y contraseña —delegada en Better Auth, conforme al diseño de seguridad— junto con las opciones de registro de cuenta nueva. El panel izquierdo comunica la propuesta del sistema y presenta la *ruta de solución*: el diagrama de estados del pipeline de generación (entrada, orquestador, cálculo, síntesis, curador y los estados terminales de aceptación y rechazo), de modo que el usuario conoce, desde el acceso, el proceso de validación iterativa que seguirá su solicitud.

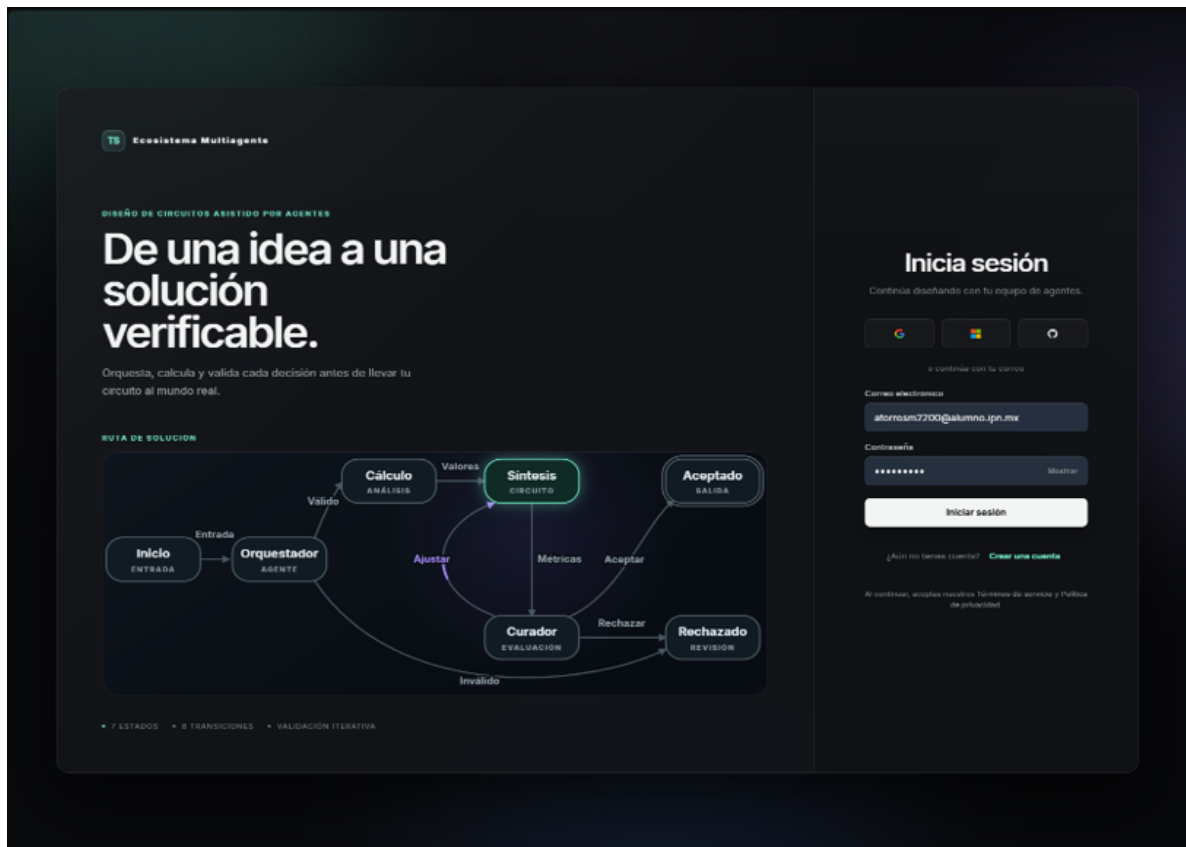


Figura 2.7. Pantalla de inicio de sesión del prototipo, con la ruta de solución del pipeline de generación.

Tras autenticarse, el usuario accede a su espacio de trabajo personal (figura 2.8). La pantalla de inicio concentra el caso de uso central del sistema: el campo de solicitud en lenguaje natural (“Describe qué quieres diseñar”), desde el cual se envía la descripción del circuito que desencadena el flujo de generación. La pantalla ofrece además un resumen operativo del consumo del periodo —tokens utilizados, costo estimado, tasa de solicitudes exitosas y tiempo procesado—, la actividad reciente con el estado de cada solicitud (completada, en proceso o que requiere atención), los proyectos que agrupan las conversaciones del usuario, y los archivos producidos por las generaciones (netlists `.cir`, resultados `.csv` y esquemáticos `.svg`). El asistente conversacional, visible en la esquina inferior, acompaña la interpretación de resultados y el ajuste de restricciones.

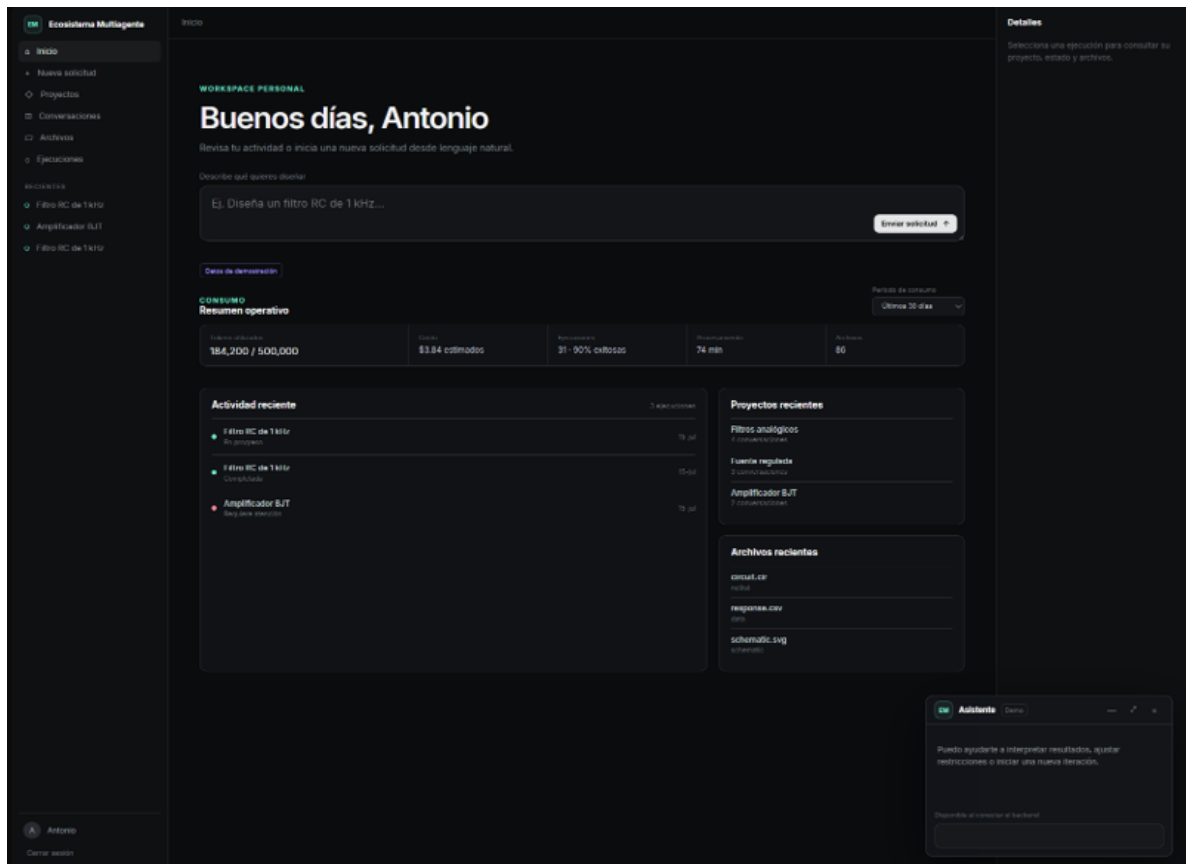


Figura 2.8. Espacio de trabajo autenticado: solicitud en lenguaje natural, resumen operativo, actividad reciente y archivos generados.

2.13. Análisis de seguridad de la implementación

El marco de seguridad de la sección 2.7 estableció los activos del sistema, los principios que ordenan su protección —confidencialidad, integridad y disponibilidad— y los controles previstos en el diseño. Esta sección analiza la implementación construida: verifica qué controles quedaron efectivamente implementados, con qué mecanismos concretos, y qué riesgos residuales permanecen. El análisis se realizó mediante revisión del código fuente del repositorio, contrastando cada control del diseño con el módulo que lo implementa.

2.13.1. Controles implementados

Autenticación y gestión de sesiones. La autenticación se delega en Better Auth, montada en el servidor bajo la ruta `/api/auth`. Las sesiones se gestionan mediante cookies configuradas según el entorno: en producción se emiten con el atributo `Secure` —que restringe su transmisión a HTTPS— y la política `SameSite` adecuada al despliegue de cliente y servidor

en orígenes distintos; en desarrollo se emplea SameSite=Lax. Un middleware de sesión resuelve el usuario en cada solicitud, y las rutas protegidas anteponen una guarda (`requireAuth`) que responde 401 ante solicitudes no autenticadas, de modo que la autorización se aplica por ruta y no queda a criterio de cada handler.

Cifrado de secretos en reposo. Las API keys de los proveedores de LLM que el administrador registra en el catálogo se cifran antes de persistirse en la base de datos con AES-256-GCM, un esquema de cifrado autenticado que protege tanto la confidencialidad como la integridad del dato: una clave alterada en la base de datos no se descifra silenciosamente, sino que falla la verificación de autenticidad. La clave de cifrado (`LLM_SECRETS_KEY`) reside únicamente en una variable de entorno. Las vistas públicas del catálogo nunca devuelven la clave descifrada; la API key solo se descifra para entregarse al subsistema de agentes por el canal interno autenticado.

Autenticación entre servicios. La comunicación entre el subsistema de agentes y el servidor —necesaria para que el orquestador obtenga la configuración activa del LLM— se realiza mediante un endpoint interno protegido por un token de servicio (`AGENTS_SERVICE_TOKEN`) transmitido como credencial *Bearer*. La verificación del token emplea comparación en tiempo constante (`timingSafeEqual`), lo que neutraliza los ataques de temporización que podrían inferir el token midiendo diferencias en el tiempo de respuesta. El subsistema de agentes no almacena la clave del LLM en disco: la conserva en una caché en memoria de vida corta (60 segundos), lo que acota la ventana de exposición.

Validación de configuración y manejo de secretos. El servidor valida la totalidad de sus variables de entorno con un esquema de Zod al arrancar y termina el proceso si la configuración es inválida o incompleta, lo que impide que el sistema opere en un estado degradado —por ejemplo, sin clave de cifrado o con un token de servicio débil (el esquema exige una longitud mínima)—. Ningún secreto se encuentra escrito en el código ni versionado en el repositorio: los archivos `.env` quedan fuera del control de versiones, en cumplimiento del requisito de manejo de secretos (RIMP-11).

Control de orígenes cruzados. El servidor aplica una política CORS con lista explícita de orígenes permitidos, leída de configuración, con soporte de credenciales. Esto impide que páginas de orígenes arbitrarios realicen solicitudes autenticadas contra la API aprovechando la sesión del usuario.

Contención de la entrada en lenguaje natural. La descripción del usuario no fluye libremente por el sistema: el orquestador exige al LLM una salida estructurada que se valida contra el esquema Pydantic de la especificación de circuito, y toda especificación —provenga del LLM o del camino estructurado— pasa por la misma validación y normalización antes de entrar al pipeline. El netlist que se simula no es texto generado libremente por el modelo, sino

que se construye de forma determinista con PySpice a partir de la especificación ya validada, lo que reduce de manera sustancial la superficie de inyección hacia la ejecución de ngspice.

2.13.2. Correspondencia con las categorías de riesgo OWASP

La tabla 2.2 resume la correspondencia entre las categorías de riesgo del OWASP Top 10 [23] relevantes para el sistema, el control implementado que las atiende y el módulo del repositorio donde puede verificarse.

Tabla 2.2. Análisis de seguridad de la implementación: riesgos, controles y evidencia en el código.

Riesgo (OWASP)	Control implementado	Evidencia (módulo)
Control de acceso roto	Guarda requireAuth por ruta; token de servicio para el canal interno	middleware/session.ts; modules/llm
Fallos criptográficos	Cifrado AES-256-GCM de API keys en reposo; cookies Secure en producción	modules/llm/llm.crypto.ts; modules/auth
Inyección	Salida estructurada del LLM validada con Pydantic; netlist generado determinísticamente con PySpice	agents/src/agents/ orquestador, llm
Fallos de identificación y autenticación	Delegación en Better Auth; comparación de token en tiempo constante	modules/auth; modules/llm/llm.index.ts
Configuración de seguridad deficiente	Validación estricta del entorno al arranque; CORS con lista de orígenes; secretos fuera del repositorio	lib/env.ts; lib/create-app.ts

2.13.3. Riesgos residuales

El análisis confirma también los límites que el marco de seguridad anticipó, y que permanecen como riesgos residuales de esta versión. El sistema no aplica control de tasa sobre las solicitudes, por lo que un usuario autenticado puede consumir recursos de cómputo y llamadas al LLM sin freno. El cifrado en reposo se limita a las API keys de los proveedores; el resto de los datos depende del cifrado que provea el servicio de base de datos. La confidencialidad de las descripciones enviadas al proveedor de LLM externo escapa al control directo del sistema. Y el aislamiento de la ejecución de ngspice descansa en la contención por contenedor del despliegue, sin un endurecimiento adicional del subproceso. Estos límites, coherentes con

el alcance de un prototipo académico, quedan señalados como trabajo futuro en materia de seguridad.

Capítulo 3

Pruebas y Análisis de Resultados

Capítulo 4

Conclusiones y Trabajo Futuro

Referencias

- [1] Z. Bao et al. «AnalogAgent: Self-Improving Analog Circuit Design Automation with LLM Agents.» arXiv: 2603.23910 [cs.AI]. dirección: <https://arxiv.org/abs/2603.23910>.
- [2] H. Zou, P. Han, E. Nazerian y A. Q. Huang. «PCBSchemaGen: Constraint-Guided Schematic Design via LLM for Printed Circuit Boards.» arXiv: 2602.00510 [cs.SE]. dirección: <https://arxiv.org/abs/2602.00510>.
- [3] K. S. Al Hasan, S. R. Raiyan, H. M. Alvee y W. Sadik. «CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts.» arXiv: 2601.04505 [cs.SE]. dirección: <https://arxiv.org/abs/2601.04505>.
- [4] Y. Lai et al., «AnalogCoder: Analog Circuit Design via Training-Free Code Generation,» en *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence (AAAI-25)*, 2025.
- [5] Z. Wang, J. Gao, W. Fu, X. Guo y X. Zhang. «AnalogSAGE: Self-evolving Analog Design Multi-Agents with Stratified Memory and Grounded Experience.» arXiv: 2512.22435 [cs.AR]. dirección: <https://arxiv.org/abs/2512.22435>.
- [6] W. Qiu, Y. Xiao y R. Pan. «HWE-Bench: Can Language Models Perform Board-level Schematic Designs?» arXiv: 2603.18102 [cs.SE]. dirección: <https://arxiv.org/abs/2603.18102>.
- [7] Y. Yin, Y. Wang, B. Xu y P. Li. «ADO-LLM: Analog Design Bayesian Optimization with In-Context Learning of Large Language Models.» arXiv: 2406.18770 [cs.SE]. dirección: <https://arxiv.org/abs/2406.18770>.
- [8] D. Vungarala, S. Alam, A. Ghosh y S. Angizi. «SPICEPilot: Navigating SPICE Code Generation and Simulation with AI Guidance.» arXiv: 2410.20553 [cs.SE]. dirección: <https://arxiv.org/abs/2410.20553>.
- [9] Ngspice Development Team, *Ngspice User's Manual*, Version 44. dirección: <https://ngspice.sourceforge.io/docs/ngspice-manual.pdf>.
- [10] LangChain. «LangGraph: Building stateful, multi-actor applications with LLMs. »dirección: <https://github.com/langchain-ai/langgraph>.

- [11] F. Salvaire, *PySpice: Simulate electronic circuit using Python and the Ngspice/Xyce simulators*, 2021. dirección: <https://github.com/PySpice-org/PySpice>.
- [12] A. Vaswani et al., «Attention Is All You Need,» en *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [13] T. B. Brown et al., «Language Models Are Few-Shot Learners,» en *Advances in Neural Information Processing Systems (NeurIPS)*, verificar lista completa de autores, 2020.
- [14] Z. Ji et al., «Survey of Hallucination in Natural Language Generation,» *ACM Computing Surveys*, vol. 55, n.º 12, 2023, verificar volumen/número y páginas.
- [15] M. Chen et al. «Evaluating Large Language Models Trained on Code.» verificar eprint y lista de autores. arXiv: 2107.03374 [cs.LG]. dirección: <https://arxiv.org/abs/2107.03374>.
- [16] R. S. Pressman, *Ingeniería del Software: Un Enfoque Práctico*, 7.^a ed. McGraw-Hill, 2010.
- [17] R. T. Fielding, «Architectural Styles and the Design of Network-based Software Architectures,» Tesis doct., University of California, Irvine, 2000.
- [18] A. S. Sedra y K. C. Smith, *Microelectronic Circuits*, 7.^a ed. Nueva York, EE. UU.: Oxford University Press, 2015, confirmar con Diego el libro de texto de referencia del dominio, ISBN: 978-0-19-933913-6.
- [19] SkyWater Technology, *SkyWater SKY130 PDK*, 2023. dirección: <https://skywater-pdk.readthedocs.io/en/main/index.html>.
- [20] ISO/IEC, *ISO/IEC 25010: Systems and software engineering — Systems and software Quality Requirements and Evaluation*, International Organization for Standardization, 2011.
- [21] W. Stallings y L. Brown, *Computer Security: Principles and Practice*, 4.^a ed. Pearson, 2018, opcional; ancla del principio CID. Alternativa: una publicación de NIST, ISBN: 978-0-13-479410-5.
- [22] Better Auth. «Better Auth Documentation.» verificar URL y año contra la versión que uses. dirección: <https://www.better-auth.com/>.
- [23] OWASP Foundation. «OWASP Top 10: The Ten Most Critical Web Application Security Risks.» verificar año de la edición que cites; 2021 es la vigente. dirección: <https://owasp.org/www-project-top-ten/>.

Apéndice A

Diagramas de casos de uso

Los siguientes diagramas complementan el modelo de casos de uso presentado en la sección 2.3.2. Cada uno detalla un subconjunto de interacciones del sistema.

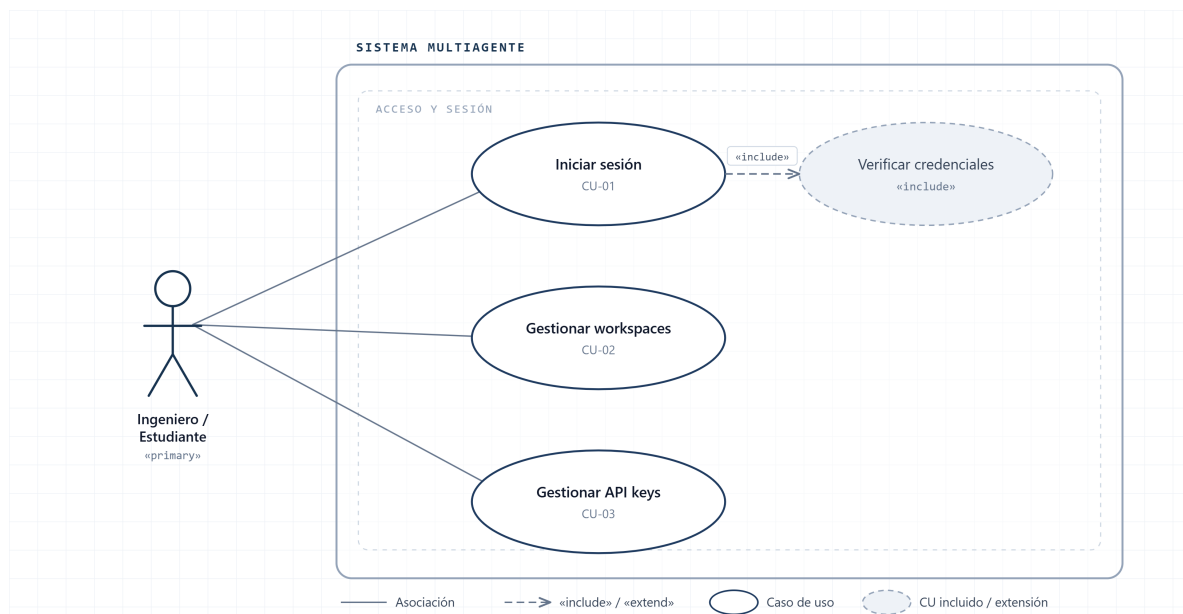


Figura A.1. Diagrama de casos de uso: acceso y gestión de sesiones.

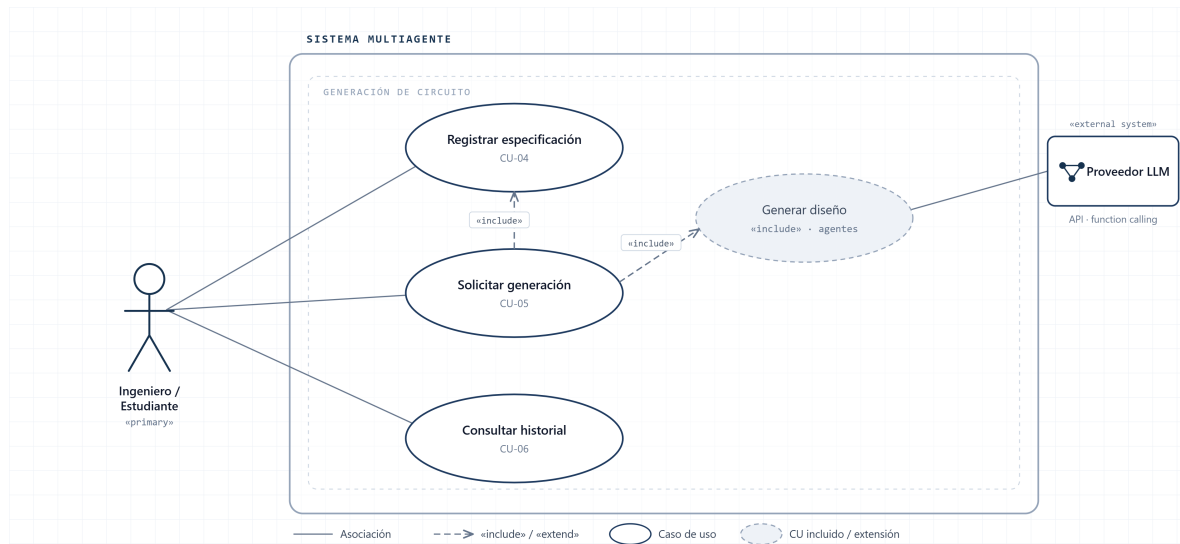


Figura A.2. Diagrama de casos de uso: generación de circuito.

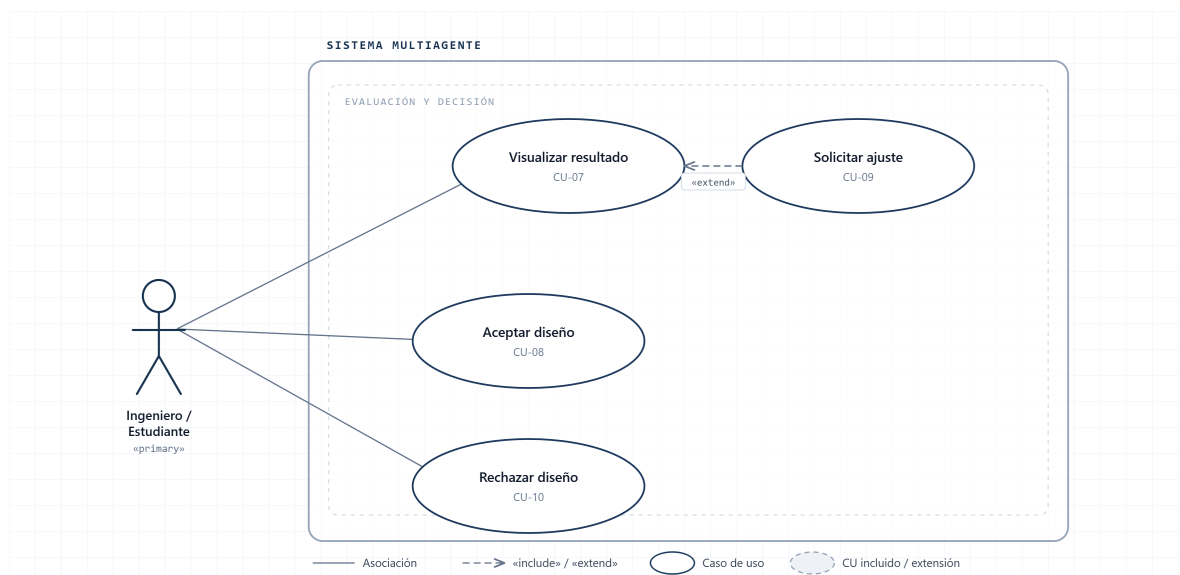


Figura A.3. Diagrama de casos de uso: decisión del agente curador.